UNIVERSITAT POLITÈCNICA DE CATALUNYA
BARCELONATECH

Escola Tècnica Superior d'Enginyeria
de Telecomunicació de Barcelona

Malware Anti-Analysis Techniques Characterization and Detection: Anti-Debugging

A Master's Thesis

Submitted to the Faculty of the

Escola Tècnica d'Enginyeria de Telecomunicació de Barcelona

Universitat Politècnica de Catalunya

by

Teo Prats

In partial fulfilment

of the requirements for the degree of

MASTER IN CYBERSECURITY

Advisor: Sebastien Kanj

Barcelona, July 2024

Abstract

This Master's Thesis investigates and compiles anti-analysis techniques used by malware developers to evade debugging, based on Cyber Threat Intelligence sources. The primary objective is to characterize these anti-debugging techniques and develop specific YARA rules to detect each one. These YARA rules will be validated using malware sample databases and subsequently shared with the cybersecurity community. The research provides a comprehensive overview of current anti-debugging strategies and offers practical tools to enhance malware detection, thereby contributing to improved protection against cyber threats.

Resumen

En este Trabajo de Fin de Máster, se investigan y recopilan las técnicas anti-análisis utilizadas por los desarrolladores de malware para evadir la depuración, basándose en fuentes de Inteligencia de Amenazas Cibernéticas. El objetivo principal es caracterizar estas técnicas anti-depuración y desarrollar reglas YARA específicas para detectar cada una de ellas. Estas reglas YARA serán validadas utilizando bases de datos de muestras de malware y posteriormente compartidas con la comunidad de ciberseguridad. La investigación proporciona una visión exhaustiva de las estrategias anti-depuración actuales y ofrece herramientas prácticas para mejorar la detección de malware, contribuyendo así a la protección contra amenazas cibernéticas.

Resum

Aquest Treball de Fi de Màster investiga i recopila les tècniques anti-anàlisi utilitzades pels desenvolupadors de malware per evadir la depuració, basant-se en fonts d'Intel·ligència d'Amenaces Cibernètiques. L'objectiu principal és caracteritzar aquestes tècniques anti-depuració i desenvolupar regles YARA específiques per detectar cadascuna d'elles. Aquestes regles YARA seran validades utilitzant bases de dades de mostres de malware i posteriorment compartides amb la comunitat de ciberseguretat. La investigació proporciona una visió integral de les estratègies anti-depuració actuals i ofereix eines pràctiques per millorar la detecció de malware, contribuint així a la protecció contra les amenaces cibernètiques.

Acknowledgements

I would like to express my deepest gratitude to all those who have supported and guided me throughout the development of this Master's Thesis.

Firstly, I extend my thanks to my project supervisor, Sebastien Kanj. His guidance and encouragement have been crucial in the successful completion of this project. He has always provided me with the best tools.

I would also like to acknowledge the support of Nuria Fernández and Francesco Mosanghini, two of the other students under the tutelage of Sebastien. Having two colleagues from the Master doing a similar work has been motivating, and we shared some ideas and comments throughout the project.

I would also like to thank Gorka Vila, his previous work “Detection of anti-analysis malware techniques: Anti-VM and Anti-Sandbox” [1] has been a great help as a basis for this project.

Their collaboration and support have been essential for the development of this project, and I am immensely grateful for their assistance and encouragement.

Revision history and approval record

Revision	Date	Purpose
1	20/04/2024	Document creation
2	13/05/2024	Document revision
3	06/06/2024	Document revision
4	28/06/2024	Document revision
6	01/07/2024	Final version

Written by:		Reviewed and approved by:	
Date	01/07/2024	Date	01/07/2024
Name	Teo Prats	Name	Sebastien Kanj
Position	Project Author	Position	Project Supervisor

Table of contents

List of Figures

List of Tables

Glossary

SEH Structured Exception Handling

PE Portable Executable

PEB Process Environment Block

PUA Potentially Unwanted Application

CPU Central Processing Unit

DLL Dynamic Link Library

OS Operating System

VM Virtual Machine

IoC Indicator of Compromise

CVE Common Vulnerabilities and Exposures

ELF Executable and Linkable Format

1. Introduction

In today's digital landscape, malware poses a significant threat to individuals and organizations alike, constantly evolving to evade detection and mitigation efforts. One of the critical areas in cybersecurity is understanding and countering the techniques used by malware developers to avoid analysis and debugging. These anti-debugging techniques are methods designed to avoid the efforts done by cybersecurity professionals to analyse and neutralize malicious software.

By developing and refining detection mechanisms, such as YARA rules, we can enhance our ability to identify and mitigate these advanced threats. This Master's Thesis focuses on investigating known anti-debugging techniques from various Cyber Threat Intelligence sources, developing specific YARA rules for each technique, and validating these rules using real-world malware samples. The ultimate goal is to contribute to the cybersecurity community by providing practical tools and insights to improve malware detection and defense mechanisms.

1.1. Context

The landscape of cybersecurity is continually evolving, with malware remaining one of the most persistent and sophisticated threats. Malware developers are constantly innovating, creating new techniques to evade detection and analysis by cybersecurity professionals. One of the critical areas of concern in malware analysis is anti-debugging techniques, which are specifically designed to thwart efforts to understand and neutralize malicious software.

In the image below [2], we can observe how the amount of different malware and PUAs are constantly growing in recent years.

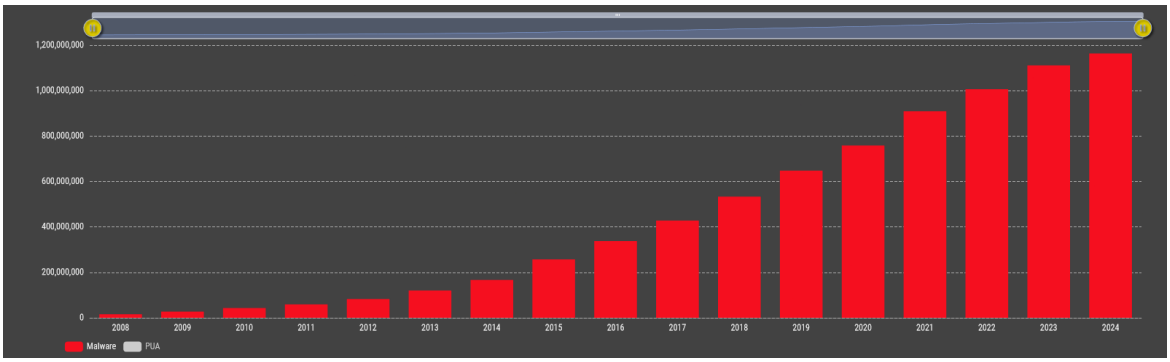

Figure 1: Total amount of malware in the lasts 16 years. Via [2]

Anti-debugging techniques encompass a wide range of methods that malware uses to detect the presence of debugging tools and alter its behaviour or terminate itself to avoid analysis. These techniques can be as simple as checking for the presence of a debugger or as complex as modifying execution paths based on detected analysis environments. Understanding these techniques is essential for cybersecurity professionals as they work to develop effective countermeasures.

In response to these challenges, tools like YARA have been developed to aid in the identification and classification of malware. YARA is a powerful tool that allows researchers to create rules based on textual or binary patterns found in malware samples. These rules help in detecting malware by looking for specific patterns or behaviours characteristic of known threats.

1.2. Objectives

The objectives of this project are the following:

- **Gather and define all known Anti-Debugging techniques**: Conduct a comprehensive review of existing literature and Cyber Threat Intelligence sources to compile a detailed list of all known anti-debugging techniques. This includes techniques employed to detect and evade debugging environments and tools.
- **Validate YARA rules using malware samples**: Test and validate the developed YARA rules against extensive databases of malware samples. This process will ensure that the rules are effective in real-world scenarios, accurately identifying malware that employs anti-debugging techniques.
- **Share findings with the cybersecurity community**: Disseminate the developed YARA rules and findings through appropriate channels within the cybersecurity community. This will include publishing the rules on relevant platforms and contributing to collaborative efforts to improve global cybersecurity resilience.

By systematically addressing these objectives this thesis aims to contribute to the ongoing efforts to protect infrastructures and sensitive information from malicious attacks.

1.3. Stakeholders

Identifying the stakeholders involved in this research is relevant and beneficial to those who have a vested interest in improving cybersecurity measures against malware. The primary stakeholders for this Master's Thesis are as follows:

- **Cybersecurity Professionals**: These include malware analysts, security researchers, and incident response teams who will directly benefit from the improved detection capabilities provided by the new YARA rules. Enhanced anti-debugging detection will aid them in more effectively identifying and mitigating malware threats.
- **Organizations and Enterprises**: Companies across various sectors, especially those handling sensitive data or critical infrastructure, such as finance, healthcare, and utilities, have a vested interest in robust cybersecurity measures. Improved detection techniques will help protect these organizations from potentially devastating cyber-attacks.
- **Academic and Research Institutions**: Researchers and students in the field of cybersecurity can utilize the insights and methodologies presented in this thesis for further study and development. The documentation of anti-debugging techniques and corresponding YARA rules can serve as a valuable resource for academic purposes.
- **Cybersecurity Community**: The broader cybersecurity community, including forums, conferences, and collaborative platforms, will benefit from the

dissemination of this research. Sharing the developed YARA rules and findings will contribute to the collective effort to strengthen global cybersecurity resilience.

1.4. Requirements and specifications

Tools and Technologies:

- **Visual Studio Code**: A versatile code editor used for writing and testing the project's source code.
- **YARA**: An essential tool in static malware analysis used to identify and classify malware based on specific patterns.
- **C++**: A programming language used to demonstrate examples of evasion techniques, providing insights into their functionality and methods for their detection.
- **GitHub**: A platform for hosting the project's source code, facilitating collaboration, and sharing findings with the community.
- **Microsoft Word**: A word processing tool used for drafting and editing sections of the project report and documentation.
- **Zotero**: reference management tool that allows users to collect, organize, cite, and share research. It facilitates automatic bibliography creation [3].

Requirements and Specifications:

- **Comprehensive Coverage**: YARA rules must cover a wide range of known anti-debugging techniques employed by malware to ensure broad detection capabilities.
- **Accuracy and Specificity**: The rules should be precisely crafted to minimize false positives and tailored to detect specific patterns, signatures, or behaviours used by malware to evade debugging.
- **Regular Updates**: An ongoing effort to update and refine YARA rules is required, based on the latest malware behaviours and advancements in anti-debugging techniques to maintain relevance and effectiveness.
- **Documented Methodology**: The creation of each YARA rule must be thoroughly documented, explaining the rationale behind it and detailing the specific anti-debugging technique it aims to detect, including examples or references to malware samples where applicable.
- **Continuous Testing and Validation**: The YARA rules should undergo rigorous testing and validation across various malware samples to ensure they effectively identify the intended anti-debugging behaviours while maintaining high accuracy and reliability.
- **Scalability and Adaptability**: YARA rules should be designed with scalability in mind, allowing them to accommodate diverse malware variants and the evolution of anti-debugging techniques over time.

1.5. Methods and procedures

This project is a fundamental part of the student's Master's research work. It follows a structured methodology to ensure systematic progress and high-quality outcomes.

- **Agile Methodology**: Tasks are broken down into short iterations, allowing for flexibility and adaptability to keep the project on track.
- **Literature Review and Data Collection**: Comprehensive review of literature and Cyber Threat Intelligence sources to gather all known anti-debugging techniques.
- **Rule Development**: Development of YARA rules based on gathered data, focusing on accuracy and specificity to detect and categorize malware behaviors effectively.
- **Testing and Refinement**: YARA rules are tested using malware sample databases such as VirusTotal and Hybrid Analysis and a local one. Continuous updates and refinements are made to improve detection capabilities and minimize false positives.
- **Documentation**: Detailed documentation includes the rationale behind each YARA rule and examples or references to malware samples, using Microsoft Word.

By following these methods and procedures, the project aims to enhance the detection and analysis of malware utilizing anti-debugging techniques, contributing valuable tools and insights to the cybersecurity field.

1.6. Work plan

The project is divided into four phases: research, development, analysis, and documentation. Each phase includes specific tasks designed to ensure timely completion and the achievement of project objectives. The detailed tasks and milestones for each phase are outlined in the following tables, accompanied by the project's Gantt chart.

Phase	Tasks	
Research	Title	Read the book Practical Malware Analysis **REF**
	Description	Read a key source of information of Anti-Debugging Techniques **REF**
	Duration	10h
Research	Title	Gather known techniques
	Description	Gather already done techniques from Anti-Debug Tricks **REF** and UnProtect Project **REF**
	Duration	50h
Research	Title	Investigate other sources
	Description	Gather information from less specific sources like **GitHub REF**
	Duration	30h
Development	Title	Edit known YARA rules
	Description	Edit if necessary already created YARA rules
	Duration	50h
Development	Title	Convert CAPA rules to YARA rules
	Description	Transform the CAPA rules found to the YARA format if possible
	Duration	5h
Development	Title	Create new YARA rules
	Description	Create new YARA rules from scratch and ideas
	Duration	40h
Testing	Title	YARA Rules testing
	Description	Test all the YARA rules gathered against a set of malware samples
	Duration	70h
Documentation	Title	Final Project documentation
	Description	Detail and explain all that has been done in the project and how
	Duration	40h

Table 1: Project's tasks. Sources: Own elaboration

Phase	Tasks	
Milestone 1	Title	Research completion
	Description	Realization of the research on Anti-Debugging techniques and YARA rules that use them
	Duration	90h
Milestone 2	Title	YARA Rules development completion
	Description	Realization of the development of YARA rules for detecting Anti-Debugging techniques
	Duration	95h
Milestone 3	Title	YARA Rules testing completion
	Description	Realization of the testing of YARA rules against the gathered malware samples
	Duration	70h
Milestone 4	Title	Final Project Documentation completion
	Description	Realization of the documentation of the project
	Duration	40h

Table 2: Project's milestones. Sources: Own elaboration

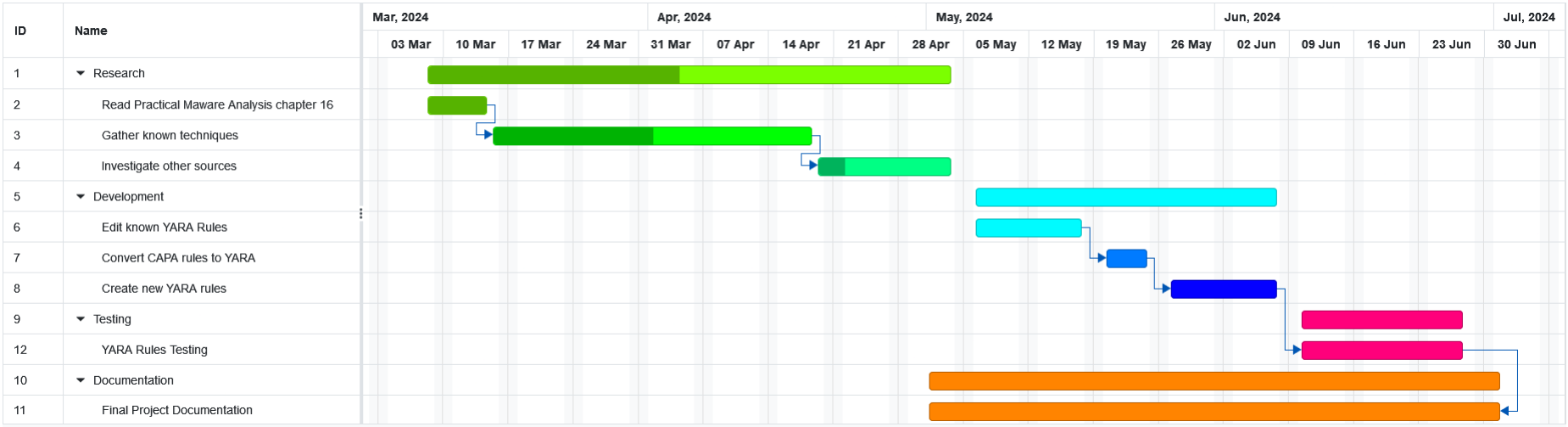

Figure 2: Gantt's chart. Via [4]. Sources: Own elaboration

2. State of the art of the technology used or applied in this thesis

In the security industry, forensic teams routinely perform in-depth analyses of malware to understand its behaviour, impact, and Indicators of Compromise (IoC), enabling them to develop more effective detection measures.

In cybersecurity, combating malware requires understanding the sophisticated techniques used by developers to evade detection. One crucial strategy is anti-debugging which we will focus on in this thesis, which employs methods like API monitoring, timing checks, and process manipulation to detect and evade debugging tools.

These techniques alter or terminate malware behaviour when a debugger is detected, complicating analysis. This section explores the current state of anti-debugging techniques, examining how they are used by malware and the existing tools for their detection. It will also focus on the development and refinement of YARA rules to identify these evasion tactics, highlighting the ongoing efforts to enhance malware detection in the cybersecurity field.

2.1. Techniques Anti-Debugging

Anti-debugging is a widely used anti-analysis technique employed by malware to detect and outsmart the use of debuggers. Malware authors are aware that analysts use debuggers to understand how malware functions, so they implement anti-debugging techniques to hide this process. When malware detects that it is running under a debugger, it may alter its execution path or modify code to cause a crash, this complicates the analysis and increases the time and effort required to understand it.

There are several Anti-debugging techniques but we are going to study the following areas Windows API, Manual checks, Object Handles, Exceptions, Timings, Breakpoints, Memory checks, Assembly instructions, Debugger interaction.

2.1.1. Windows API

The Windows API offers several functions that a program can use to detect if it is being debugged. Some of these functions are specifically designed for debugger detection, while others, originally intended for different purposes, can be repurposed to identify a debugger. Additionally, some functions use undocumented API features.

The simplest way to bypass calls to these anti-debugging API functions is to manually alter the malware during execution, either by preventing it from calling these functions or by modifying the flags after the call to ensure the correct execution path is followed. A more complex approach would be to hook these functions, similar to how a rootkit operates.

Some functions examples would be: *IsDebuggerPresent*, *CheckRemoteDebuggerPresent*, *NtQueryInformationProcess*, *OutputDebugString*

2.1.2. Manual checks

While using the Windows API is a straightforward method for detecting the presence of a debugger, manually inspecting system structures is the most prevalent technique used by malware authors. There are several reasons why malware developers avoid relying on the Windows API for anti-debugging. For instance, API calls can be hooked by rootkits to return false information. As a result, malware authors often opt to manually perform the equivalent of these API calls to ensure more reliable detection.

When performing manual checks, various flags within the Process Environment Block (PEB) structure can reveal the presence of a debugger.

2.1.3. Identifying Debugger Behaviour

Debuggers are often used to set breakpoints or single-step through a process to assist malware analysts in reverse-engineering. However, these operations modify the code within the process. Malware employs several anti-debugging techniques to detect such debugger behaviour basically knowing how the debugger acts when the CPU executes a certain instruction.

2.1.4. Timing

Timing checks are a popular method for malware to detect debuggers because processes run more slowly when being debugged. For example, single-stepping through a program significantly slows down execution.

There are a couple of ways to use timing checks to detect a debugger:

- **Timestamp Comparison**: Record a timestamp, perform a series of operations, and then take another timestamp. Compare the two timestamps; if there is a noticeable lag, a debugger is likely present.
- **Exception Timing**: Take a timestamp before and after raising an exception. In a non-debugged process, the exception is handled quickly, while a debugger handles exceptions much more slowly. Most debuggers require human intervention to manage exceptions, causing significant delays. Even if the debugger is set to ignore exceptions and pass them to the program, there will still be a noticeable delay.

2.1.5. Exceptions

The interrupts generate exceptions that debuggers use for operations like breakpoints.

Exceptions can be utilized to disrupt or detect a debugger. Most exception-based detection methods rely on the fact that debuggers will trap exceptions and not immediately pass them to the process being debugged. By default, most debuggers trap exceptions instead of passing them directly to the program. If the debugger fails to pass the exception to the process properly, this can be detected within the process's exception-handling mechanism.

2.1.6. Inserting interrupts

A classic anti-debugging technique involves using exceptions to frustrate the analyst and disrupt normal program execution by inserting interrupts within a valid instruction sequence. Depending on the debugger's settings, these interrupts can cause the

debugger to stop, as they utilize the same mechanism that the debugger uses to set software breakpoints.

And there are more subtypes and different classifications but those are the principal ones.

2.2. YARA rules

YARA is an open-source tool developed by Victor Alvarez, a software engineer and antivirus researcher at VirusTotal [5], primarily used for the static analysis of malware. It aids malware researchers in identifying and classifying malware samples.

YARA rules are written in a simple, easy-to-understand syntax with the extension .yara. They define variables containing textual or binary patterns, such as strings, hexadecimal codes, or regular expressions, found in malware samples. If certain conditions within the rule are met, the rule can successfully identify a piece of malware.

When analysing malware, researchers identify unique patterns and strings that attribute the sample to a specific threat group or malware family. By creating a YARA rule based on several samples from the same malware family, it's possible to identify multiple samples associated with the same campaign or threat actor.

YARA has various use cases, including identifying and classifying malware, finding new samples based on family-specific patterns, identifying compromised devices for incident responders, and deploying custom YARA rules for proactive defenses within an organization.

However, YARA's limitation is that it is a static analysis tool, meaning it can only detect malware based on predefined patterns and strings. It cannot detect obfuscated or encrypted malware, as these patterns and strings will be hidden from the YARA rule.

The structure of YARA is the following:

- **Meta**: The *meta* section contains descriptive information about the rule, this section contains the next fields:
    - **Author**: The author or authors of the rule.
    - **Description**: A brief and clear description of what the rule seeks to detect.
    - **Date**: The date the rule was created or last modified.
    - **Reference**: Additional references relevant to the rule, such as articles, technical documents, or CVEs.
    - **Hash**: A hash associated with the rule, such as MD5, SHA-1, or SHA-256.
- **Strings**: This is the most important section; this section defines the patterns that the rule will search for in the analysed files. These patterns can be text strings, hexadecimal sequences, or regular expressions. Now I'm going to stand out the most important syntaxis.
    - **nocase**: Indicates whether the search for the string is case-insensitive.
    - **wide**: Indicates whether the string should be interpreted as a wide character string (UTF-16).
    - **ascii**: Indicates whether the string should be interpreted as an ASCII character string.

  - **fullword**: Indicates that the string should match a whole word and not part of a word.
  - **private**: Indicates that the string should be treated as confidential and not included in scan reports.

- **Conditions**:
  - **Logical operators**: These can be used to combine the boolean expressions that define the conditions of the rule (*AND, OR, NOT*).
  - **Functions**: Some special YARA functions can be used within the conditions, such as *all(), any(), all of them(), any of them().*

- **Imports**: External modules: Imported modules extend the functionality of YARA and allow access to additional features, such as analysis of PE or ELF files.

3. Methodology / project development

The Methodology is included in this chapter and should include all relevant methods that have been used as well as research methods and measurements, software and hardware development, etc.

3.1. Research of techniques

For the research of Anti-Debugging techniques, I mainly checked these sources of information:

- **Check Point Research**: Check Point Research is a leading provider of cyber threat intelligence, delivering insights and analysis into the latest vulnerabilities, exploits, and malware trends. Their reports and blog posts often contain in-depth analyses of various malware families and their techniques, including Anti-Debugging methods [6].
- **Unprotect Project**: The Unprotect Project is a collaborative effort aimed at sharing knowledge and tools related to reverse engineering, malware analysis, and software protection techniques. It hosts a repository of resources, articles, and tools contributed by security researchers and practitioners worldwide, making it a valuable source for information on Anti-Debugging. This was indeed the principal source of YARA rules [7].
- **GitHub developers**: GitHub is a popular platform for hosting and collaborating on software projects, including security-related tools and research. Many developers and security researchers share their code, scripts, and findings related to Anti-Debugging techniques on GitHub [8].
- **Practical Malware Analysis**: Practical Malware Analysis: "Practical Malware Analysis" is a comprehensive guidebook written by Michael Sikorski and Andrew Honig, which covers various aspects of malware analysis, including Anti-Debugging techniques. The book provides hands-on exercises and real-world examples to help readers understand how malware works and how to analyse it effectively, making it a valuable resource for researchers and practitioners alike. I am going to focus on the chapter 16 which is where explains the Anti-Debugging techniques [9].

3.2. Documentation of techniques

The techniques will be documented in the following way:

- **Technique 1**: **Name**: name of the technique.
- **Description**: brief description of how the YARA works and what is the idea behind.
- **YARA rule**: a reference to the code snippet with the implemented YARA rule in the appendix section alongside the done tests.
- **References**: links to the sources of information.
- **Comments**: in some rules a brief commentary about the performance of the rule.

3.3. Understanding of YARA

For the understanding of YARA rules mainly I used the official documentation of the tool [10] that is provided by VirusTotal [5].

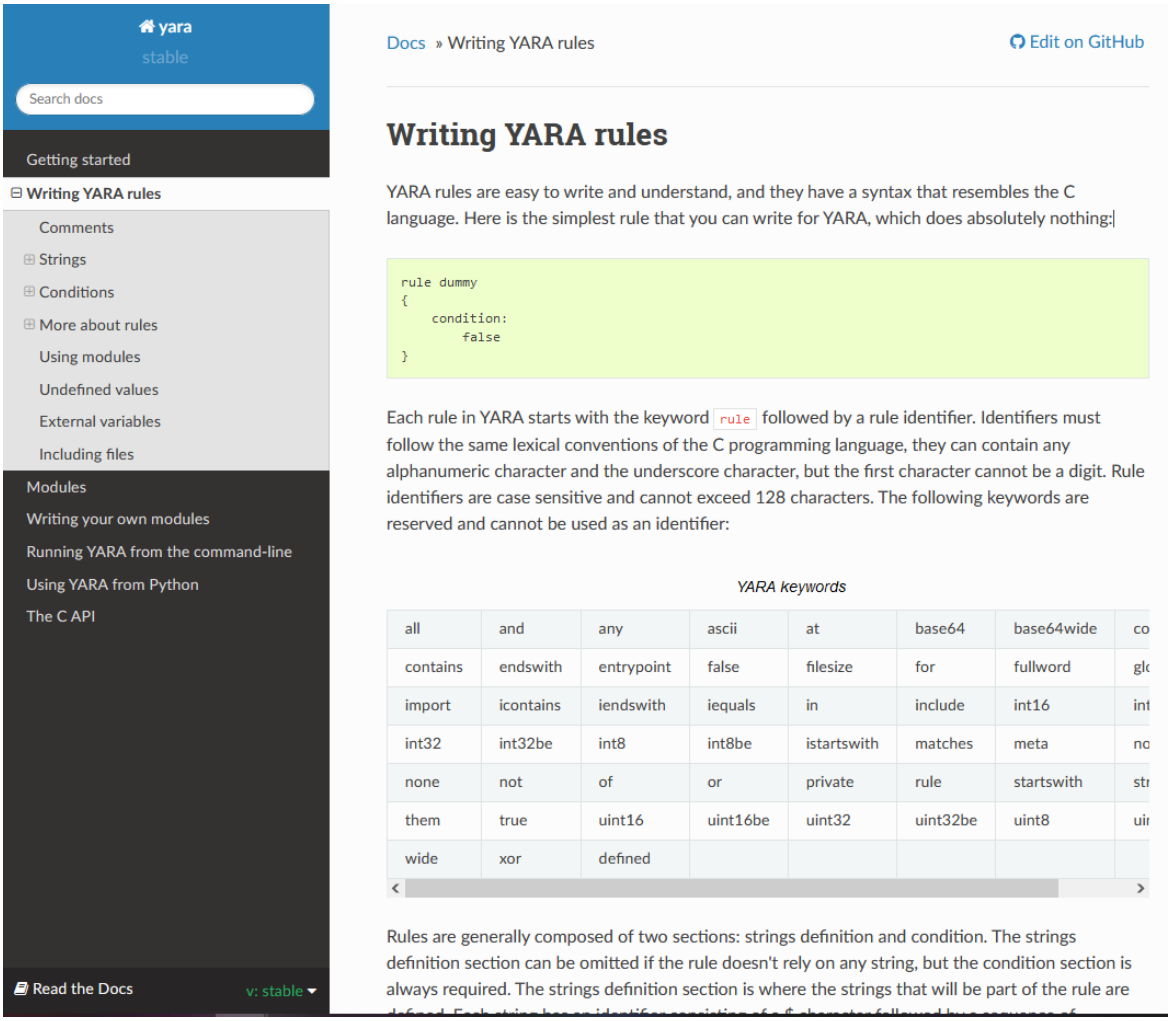

Figure 3: YARA official documentation [10]

Using this documentation, I was able to understand the structure of the rules, their modules, conditions, strings and all the important information in order to work with YARA.

3.4. Creation of YARA rules

For the creation of YARA rules we have three scenarios which are the following:

- **YARA already created:** this is the easiest scenario, in this case I am going to analyse the YARA already created, understand and edit it if it is needed in order to upgrade the detection capability.
- **CAPA rule created**: CAPA rules are other type of rules similar to YARA but a bit more complex, I found some samples that doesn't have the equivalent YARA rule so the plan is to obtain a YARA from the CAPA. For this method I did a huge research about the CAPA rules in forums [11] and different GitHub's [12]

- **Found ideas**: In some cases, I found ideas explained with a detailed description and an example with a code snippet but that does not have their YARA rule, in this case I am going to understand the given examples and create the corresponding YARA rule. Almost all the ideas were taken from 'Anti-Debug Tricks' [6]

To test that all the YARA rules are valid I am going to use the online tool YARA TOOLKIT [13], this tool first verifies that the rule syntaxis is valid then compiles it and allows you to download it.

Figure 4: YARA toolkit [13]

3.5. Testing the YARA rules

For the testing the methodology will be the next:

Before start, I have to comment that in the early stages of the project the main idea was to run every YARA rule in 'Hybrid Analysis' [14], a website that allows you to test the YARA rules against their malware database but after doing some tries we decided to desist due to some inconsistency in some of the output when we executed it.

How Hybrid Analysis did not work we chose to download a huge number of samples and test the rules in local, the samples have been downloaded from 'Virus Share' [15], a huge repository of malware samples.

We decided to deal for the moment with windows malware to test the YARA rules because Windows is the most widely used operating system and, consequently, the most targeted by malware developers. This approach allows a more comprehensive evaluation of the YARA rules effectiveness in detecting real-world threats that impact the majority of users. Moreover, concentrating on a single operating system simplifies the testing process and ensures more consistent and reliable results.

Once we have the malware folder ready and the YARA rule to test, we are going to execute it using Visual Studio Code [16]:

This is the rule example, this one works with the possibility of use the *NtSetInformationThread* function as a way to hide threads from debuggers.

rule.yar

```
1   rule Detect_NtSetInformationThread: AntiDebug {
2       meta:
3           description = "Detect NtSetInformationThread as anti-debug"
4           author = "Unprotect"
5           comment = "Experimental rule"
6       strings:
7           $1 = "NtSetInformationThread" fullword ascii
8       condition:
9           $1
10  }
```

Figure 5: YARA rule example

So, if we execute the YARA rule over the malware folder we obtain some detections

```
PS C:\Users\Murakami Fanboy\Desktop\yara> .\yara64.exe -r rule.yar malware
Detect_NtSetInformationThread malware\VirusShare_014a9cb92514e27c0107614df764bc06
Detect_NtSetInformationThread malware\VirusShare_070d303c95856722bea316d01b42df46
Detect_NtSetInformationThread malware\VirusShare_13f90b323b0acdece64fe7f41126e675
Detect_NtSetInformationThread malware\VirusShare_1574ee69c6445bf8e0fd26f2e9a703c0
Detect_NtSetInformationThread malware\VirusShare_260a7503f7d26636ee08fe2ce1264c47
```

Figure 6: Example of malware detection using YARA

As we can appreciate in the image below, the YARA rule works as expected and is able to detect the malware.

Is crucial to comment that we are running this on a folder full of malware, without normal code, so we are not going to be able how many false positives a rule can get. In this case the NtSetInformationThread is used in 23/997 samples of malware but could be used too in a non-malicious way.

4. Results

In this section will be displayed all the YARA rules recollected from the different sources of information mentioned previously in addition to those created from the ideas found.

The structure of the rules will be displayed as mentioned in the Methodology section, every rule will have the code snippet and the testing done referenced in the appendix section.

4.1. Technique 1: IsDebuggerPresent

- **Description**: This function checks specific flag in the Process Environment Block (PEB) for the field IsDebugged which will return zero if the process is not running into a debugger or a nonzero if a debugger is attached.

  If you want to understand the underlying process of IsDebuggerPresent API you can check the code snippet section for the following method: IsDebugged Flag.

- **YARA Rule**: See the YARA rule for this technique in the subsection A.1.1
- **Results**: This YARA rule detected 314/997 malware files.
- **References**:
  - https://anti-debug.checkpoint.com/techniques/debug-flags.html
  - https://unprotect.it/technique/isdebuggerpresent/

4.2. Technique 2: NtQueryInformationProcess

- **Description**: This function retrieves information about a running process. Malware are able to detect if the process is currently being attached to a debugger using the ProcessDebugPort (0x7) information class.

  A nonzero value returned by the call indicates that the process is being debugged.

- **YARA Rule**: See the YARA rule for this technique in the subsection A.1.2
- **Results**: This YARA rule detected 35/997 malware files.
- **References**:
  - https://unprotect.it/technique/ntqueryinformationprocess/
  - https://anti-debug.checkpoint.com/techniques/debug-flags.html

4.3. Technique 3: NtSetInformationThread

- **Description**: NtSetInformationThread can be used to hide threads from debuggers using the ThreadHideFromDebugger ThreadInfoClass (0x11 / 17). This is intended to be used by an external process, but any thread can use it on itself.

  After the thread is hidden from the debugger, it will continue running but the debugger won't receive events related to this thread. This thread can perform anti-debugging checks such as code checksum, debug flags verification, etc.

- **YARA Rule**: See the YARA rule for this technique in the subsection A.1.3

- **Results**: This YARA rule detected 23/997 malware files.
- **References**:
  - https://anti-debug.checkpoint.com/techniques/interactive.html
  - https://unprotect.it/technique/ntsetinformationthread/

4.4. Technique 4: NtQueryObject

- **Description**: This function retrieves object information. By calling this function with the class ObjectTypeInformation will retrieve the specific object type (debug) to detect the debugger.
- **YARA Rule**: See the YARA rule for this technique in the subsection A.1.4
- **Results**: This YARA rule detected 26/997 malware files.
- **References**:
  - https://unprotect.it/technique/ntqueryobject/
- **Comments**: in this rule for example, if I only I had used the wide option I would have got 2 malware samples, and if I only had used ascii I would have got 24 samples but using both options I got 26.

4.5. Technique 5: OutputDebugString

- **Description**: This Windows API is often used by developers for debugging purpose. It will display a text to the attached debugger. This API is also used by Malware to open a communication channel between one or multiple processes.

  It is possible to use OutputDebugString in addition of GetLastError / SetLastError to detect debugger presence.
- **YARA Rule**: See the YARA rule for this technique in the subsection A.1.5
- **Results**: This YARA rule detected 221/997 malware files.
- **References**:
  - https://unprotect.it/technique/outputdebugstring/
- **Comments**: originally this YARA rules only checked the OutputDebugStringA, in this way 136 malware samples were detected but if we added the OutputDebugStringW (the wide option) along with the Ascii version we get 85 more malware samples.

4.6. Technique 6: EventPairHandles

- **Description**: An EventPair Object is an event constructed by two _KEVENT structures which are conventionally named High and Low.

  There is a relation between generic Event Objects and Debuggers because they must create a custom event called DebugEvent able to handle exceptions. Due to the presence of events owned by the Debugger, every information relative to the events of a normal process differs from a debugged process.
- **YARA Rule**: See the YARA rule for this technique in the subsection A.1.6

- **Results**: This YARA rule detected 1/997 malware files.

- **References**:
  - https://unprotect.it/technique/eventpairhandles/

4.7. Technique 7: CsrGetProcessID

- **Description**: This function is undocumented within *OpenProcess*. It can be used to get the PID of CRSS.exe, which is a SYSTEM process. By default, a process has the *SeDebugPrivilege* privilege in their access token disabled.

  However, when the process is loaded by a debugger such as OllyDbg or WinDbg, the *SeDebugPrivilege* privilege is enabled. If a process is able to open CRSS.exe process, it means that the process *SeDebugPrivilege* enabled in the access token, and thus, suggesting that the process is being debugged.

  If we call *OpenProcess* and pass the ID returned by *CsrGetProcessId*, no error will occur if the *SeDebugPrivilege* has been set with *SetPrivilege* / *AdjustTokenPrivileges*.
- **YARA Rule**: See the YARA rule for this technique in the subsection A.1.7
- **Results**: This YARA rule detected 0/997 malware files.
- **References**:
  - https://unprotect.it/technique/csrgetprocessid/
- **Comments**: if we search *CsrGetProcessID* without *fullword* we obtain 653 samples but the rule becomes so much generic.

4.8. Technique 8: CloseHandle, NtClose

- **Description**: When a process is debugged, calling NtClose or CloseHandle with an invalid handle will generate a STATUS_INVALID_HANDLE exception.

  The exception can be cached by an exception handler. If the control is passed to the exception handler, it indicates that a debugger is present.
- **YARA Rule**: See the YARA rule for this technique in the subsection A.1.8
- **Results**: This YARA rule detected 685/997 malware files.
- **References**:
  - https://unprotect.it/technique/closehandle-ntclose/

4.9. Technique 9: NtGlobalFlag

- **Description**: The information that the system uses to determine how to create heap structures is stored at an undocumented location in the PEB at offset 0x68. If the value at this location is 0x70, we know that we are running in a debugger.

  The NtGlobalFlag field of the Process Environment Block (0x68 offset on 32-Bit and 0xBC on 64-bit Windows) is 0 by default. Attaching a debugger doesn't

change the value of NtGlobalFlag. However, if the process was created by a debugger, the following flags will be set:

FLG_HEAP_ENABLE_TAIL_CHECK (0x10)

FLG_HEAP_ENABLE_FREE_CHECK (0x20)

FLG_HEAP_VALIDATE_PARAMETERS (0x40)

The presence of a debugger can be detected by checking a combination of those flags.

- **YARA Rule**: See the YARA rule for this technique in the subsection A.1.9
- **Results**: This YARA rule detected 0/997 malware files.
- **References**:
  - https://unprotect.it/technique/ntglobalflag/

4.10. Technique 10: RDTSC

- **Description**: The Read-Time-Stamp-Counter (RDTSC) instruction can be used by malware to determine how quickly the processor executes the program's instructions. It returns the count of the number of ticks since the last system reboot as a 64-bit value placed into EDX:EAX.

  It will execute RDTSC twice and then calculate the difference between low order values and check it with CMP condition. If the difference lays below 0FFFh no debugger is found if it is above or equal, then application is debugged.
- **YARA Rule**: See the YARA rule for this technique in the subsection A.1.10
- **Results**: This YARA rule detected 455/997 malware files.
- **References**:
  - https://unprotect.it/technique/rdtsc/
  - https://anti-debug.checkpoint.com/techniques/timing.html

4.11. Technique 11: Detecting Window with FindWindow API

- **Description**: The FindWindowA / FindWindowW function can be used to search for windows by name or class.

  It is also possible to use EnumWindows API in conjunction with GetWindowTextLength and GetWindowText to locate a piece of string that could reveal the presence of a known debugger.
- **YARA Rule**: See the YARA rule for this technique in the subsection A.1.11
- **Results**: This YARA rule detected 0/997 malware files.
- **References**:
  - https://unprotect.it/technique/detecting-window-with-findwindow-api/

4.12. Technique 12: Detecting Running Process: EnumProcess API

- **Description**: Anti-monitoring is a technique used by malware to prevent security professionals from detecting and analyzing it. One way that malware can accomplish this is by using the EnumProcess function to search for specific processes, such as ollydbg.exe or wireshark.exe, which are commonly used by security professionals to monitor and analyze running processes on a system.

  By detecting these processes and taking evasive action, such as terminating itself or encrypting its own code, malware can prevent security professionals from gaining visibility into its activities and disrupt their efforts to analyze it.

- **YARA** Rule: See the YARA rule for this technique in the subsection A.1.12
- **Results**: This YARA rule detected 121/997 malware files.
- **Comments**: This YARA originally didn't use the wide option, adding it the YARA detected 2 more samples.
- **References**:
  - https://unprotect.it/technique/detecting-running-process-enumprocess-api/

4.13. Technique 13: TLS Callback

- **Description:** TLS (Thread Local Storage) callbacks are a mechanism in Windows that allows a program to define a function that will be called when a thread is created. These callbacks can be used to perform various tasks, such as initializing thread-specific data or modifying the behavior of the thread.

  As an anti-debugging technique, a program can use a TLS callback to execute code before the main entry point of the program, which is defined in the PE (Portable Executable) header. This allows the program to run secretly in a debugger, as the debugger will typically start at the main entry point and may not be aware of the TLS callback.

  The program can use the TLS callback to detect whether it is being debugged, and if it is, it can terminate the process or take other actions to evade debugging. This technique can be used to make it more difficult for a debugger to attach to the process and to hinder reverse engineering efforts.

- **YARA Rule**: See the YARA rule for this technique in the subsection A.1.13
- **Results**: This YARA rule detected 2/997 malware files.
- **References**:
  - https://unprotect.it/technique/tls-callback/

4.14. Technique 14: Bad String Format

- **Description:** Bad string format is a technique used by malware to evade detection and analysis by OllyDbg, a popular debugger used by security researchers and analysts. This technique involves using malformed strings that exploit a known bug in OllyDbg, causing the debugger to crash or behave unexpectedly.

For example**,** the malware may use a string with multiple %s inputs, which OllyDbg is not able to handle correctly. This causes the debugger to crash or behave in an unpredictable manner, making it difficult for the analyst to continue their analysis. This technique can be effective in disrupting the analysis process and making it more difficult for the analyst to understand the malware's capabilities and behavior. However, it is only effective against OllyDbg, and other debuggers may not be affected by this technique.

- **YARA Rule**: See the YARA rule for this technique in the subsection A.1.14
- **Results**: This YARA rule detected 32/997 malware files.
- **References**:
    - https://unprotect.it/technique/bad-string-format/

4.15. Technique 15: Unhandled Exception Filter

- **Description**: An application-defined function that passes unhandled exceptions to the debugger, if the process is being debugged. Otherwise, it optionally displays an application error message box and causes the exception handler to be executed.

  If an exception occurs and no exception handler is registered, the UnhandledExceptionFilter function will be called. It is possible to register a custom unhandled exception filter using the SetUnhandledExceptionFilter. But if the program is running under a debugger, the custom filter won't be called, and the exception will be passed to the debugger.

  Therefore, if the unhandled exception filter is registered and the control is passed to it, then the process is not running with a debugger.
- **YARA Rule**: See the YARA rule for this technique in the subsection A.1.15
- **Results**: This YARA rule detected 646/997 malware files.
- **References**:
    - https://unprotect.it/technique/unhandled-exception-filter/
    - https://anti-debug.checkpoint.com/techniques/exceptions.html

4.16. Technique 16: Interrupts

- **Description:** Adversaries may use exception-based anti-debugging techniques to detect whether their code is being executed in a debugger. These techniques rely on the fact that most debuggers will trap exceptions and not immediately pass them to the process being debugged for handling.

  By triggering an exception and checking whether it is handled properly, the adversary's code can determine whether it is being executed in a debugger and take appropriate action, such as exiting or altering its behaviour. This can be achieved using interrupt instructions such as INT 3 or UD2 to trigger the exception. This technique can be used to evade detection and make reverse engineering more difficult.
- **YARA Rule**: See the YARA rule for this technique in the subsection A.1.16

- **Results**: This YARA rule detected 992/997 malware files.
- **References**:
  - https://unprotect.it/technique/interrupts/

4.17. Technique 17: INT3 Instruction Scanning

- **Description**: Instruction INT3 is an interruption which is used as Software breakpoints. These breakpoints are set by modifying the code at the target address, replacing it with a byte value 0xCC (INT3 / Breakpoint Interrupt).

  The exception EXCEPTION_BREAKPOINT (0x80000003) is generated, and an exception handler will be raised. Malware identify software breakpoints by scanning for the byte 0xCC in the protector code and/or an API code.
- **YARA Rule**: See the YARA rule for this technique in the subsection A.1.17
- **Results**: This YARA rule detected 997/997 malware files.
- **References**:
  - https://unprotect.it/technique/int3-instruction-scanning/
  - https://unprotect.it/technique/ice-0xf1/
  - https://unprotect.it/technique/int-0x2d/
  - https://anti-debug.checkpoint.com/techniques/process-memory.html#breakpoints
- **Comments**: The execution of this rule is very slow due to the defined strings, they are single byte strings so YARA needs to scan each byte of the files.

4.18. Technique 18: NtSetDebugFilterState

- **Description:** The NtSetDebugFilterState and DbgSetDebugFilterState functions are used by malware to detect the presence of a kernel mode debugger. These functions allow the malware to set up a debug filter, which is a mechanism that can be used to detect and respond to the presence of a debugger.

  When a kernel mode debugger is present, the debug filter will be triggered, and the malware can then take actions to evade detection and continue to operate. This technique is commonly used by malware to avoid analysis by security researchers and avoid being detected by security software. By using these functions, the malware can operate stealthily and evade detection, making it difficult for analysts to reverse engineer the malware and understand its capabilities and behaviors.
- **YARA Rule**: See the YARA rule for this technique in the subsection A.1.18
- **Results**: This YARA rule detected 0/997 malware files.
- **References**:
  - https://unprotect.it/technique/ntsetdebugfilterstate/

4.19. Technique 19: Guard Pages

- **Description:** Memory breakpoints are a technique used by malware to detect if a debugger is present. This technique involves setting up a "guard page" in memory, which is a page of memory that is protected by the operating system and cannot be accessed by normal code. If a debugger is present, the malware can use this guard page to detect its presence.

  This technique works by putting a return address onto the stack, then accessing the guard page. If the operating system detects that the guard page has been accessed, it will raise a STATUS_GUARD_PAGE_VIOLATION exception. The malware can then check for this exception, and if it is present, it can assume that no debugging is taking place. This allows the malware to evade detection and continue to operate without being interrupted by a debugger.

- **YARA Rule**: See the YARA rule for this technique in the subsection A.1.19
- **Results**: This YARA rule detected 53/997 malware files.
- **References**:
  - https://unprotect.it/technique/guard-pages/
- **Comments**: If we lower the condition from 4 to 2 for example we detect 211 samples, but at the same time we are being more permissive and we could have more false positives.

4.20. Technique 20: SuspendThread

- **Description:** Suspending threads is a technique used by malware to disable user-mode debuggers and make it more difficult for security analysts to reverse engineer and analyze the code. This can be achieved by using the SuspendThread function from the kernel32.dll library or the NtSuspendThread function from the NTDLL.DLL library.

  The malware can enumerate the threads of a given process, or search for a named window and open its owner thread, and then suspend that thread. This will prevent the debugger from running and make it more difficult to analyze the code.

  This technique can be used by malware authors to evade detection and analysis, and make their code more difficult to understand.

- **YARA Rule:** See the YARA rule for this technique in the subsection A.1.20
- **Results**: This YARA rule detected 1/997 malware files.
- **References**:
  - https://unprotect.it/technique/suspendthread/
- **Comments**: In the original YARA there were a few syntactic issues and I had to guess what was the intention.

4.21. Technique 21: LocalSize(0)

- **Description:** The function LocalSize retrieves the current size of the specified local memory object, in bytes. By setting the hMem parameters with 0 will trigger an exception in a debugger that can be used as an anti-debugging mechanism.
- **YARA Rule**: See the YARA rule for this technique in the subsection A.1.21
- **Results**: This YARA rule detected 7/997 malware files.
- **References**:
  - https://unprotect.it/technique/localsize0/

4.22. Technique 22: CheckRemoteDebuggerPresent

- **Description:** This rule looks for API function calls that malicious programs use to determine if they are being debugged.
- **YARA Rule**: See the YARA rule for this technique in the subsectionA.1.22
- **Results**: This YARA rule detected 1/997 malware files.
- **References**:
  - https://unprotect.it/technique/checkremotedebuggerpresent/
  - https://anti-debug.checkpoint.com/techniques/debug-flags.html
- **Comments**: The only sample malware detected with this YARA meets the 4 strings conditions.

4.23. Technique 23: IsDebugged Flag

- **Description:** While a process is running, the location of the PEB can be referenced by the location fs:[30h]. For anti-debugging, malware will use that location to check the BeingDebugged flag, which indicates whether the specified process is being debugged.
- **YARA Rule**: See the YARA rule for this technique in the subsection A.1.23
- **Results**: This YARA rule detected 0/997 malware files.
- **References**:
  - https://unprotect.it/technique/isdebugged-flag/

4.24. Technique 24: GetTickCount, GetLocalTime, GetSystemTime, timeGetTime

- **Description:** This is typical timing function which is used to measure time needed to execute some function/instruction set. If the difference is more than fixed threshold, the process exits.

  GetTickCount reads from the KUSER_SHARED_DATA page. This page is mapped read-only into the user mode range of the virtual address and read-write in the kernel range. The system clock tick updates the system time, which is stored directly in this page.
- **YARA Rule**: See the YARA rule for this technique in the subsection A.1.24

- **Results**: This YARA rule detected 665/997 malware files.
- **References**:
  - https://unprotect.it/technique/gettickcount/
  - https://anti-debug.checkpoint.com/techniques/timing.html
  - https://unprotect.it/technique/getlocaltime-getsystemtime-timegettime-ntqueryperformancecounter/
  - https://anti-debug.checkpoint.com/techniques/timing.html

4.25. Technique 25: Debug Registers, Hardware Breakpoints

- **Description:** Registers DR0 through DR3 contain the linear address associated with one of the four hardware breakpoint conditions. For anti-debugging, malware will check the contents of the first four debug registers to see if the hardware breakpoint has been set.
- **YARA Rule**: See the YARA rule for this technique in the subsection A.1.25
- **Results**: This YARA rule detected 19/997 malware files.
- **References**:
  - https://unprotect.it/technique/debug-registers-hardware-breakpoints/
  - https://anti-debug.checkpoint.com/techniques/process-memory.html#anti-step-over

4.26. Technique 26: Trap Flag

- **Description:** There is a Trap Flag in the Flags register. Bit number 8 of the EFLAGS register is the trap flag. When the Trap Flag is set, a SINGLE_STEP exception is generated.
- **YARA Rule**: See the YARA rule for this technique in the subsectionA.1.26
- **Results**: This YARA rule detected 22/997 malware files.
- **References**:
  - https://unprotect.it/technique/trap-flag/

4.27. Technique 27: NtYieldExecution() / SwitchToThread()

- **Description:** his method is not really reliable because it only shows if there a high priority thread in the current process. However, it could be used as an anti-tracing technique.

  When an application is traced in a debugger and a single-step is executed, the context can't be switched to other thread. This means that ntdll!NtYieldExecution() returns STATUS_NO_YIELD_PERFORMED (0x40000024), which leads to kernel32!SwitchToThread() returning zero.

The strategy of using this technique is that there is a loop which modifies some counter if kernel32!SwitchToThread() returns zero, or ntdll!NtYieldExecution() returns STATUS_NO_YIELD_PERFORMED. This can be a loop which decrypts strings or some other loop which is supposed to be analysed manually in a debugger. If the counter has the expected value (expected i.e. the value if all kernel32!SwitchToThread() returned zero) after leaving the loop, we consider that the debugger is present.

- **YARA Rule**: See the YARA rule for this technique in the subsection A.1.27
- **Results**: This YARA rule detected 99/997 malware files.
- **References**:
  - https://anti-debug.checkpoint.com/techniques/misc.html

4.28. Technique 28: VirtualAlloc() / GetWriteWatch()

- **Description:** This technique was described as a suggestion for a famous al-khaser solution, a tool for testing VMs, debuggers, sandboxes, AV, etc. against many malware-like defences.

  The idea is drawn from the documentation for GetWriteWatch function where the following is stated in a “Remarks” section:

  “When you call the VirtualAlloc function to reserve or commit memory, you can specify MEM_WRITE_WATCH. This value causes the system to keep track of the pages that are written to in the committed memory region. You can call the GetWriteWatch function to retrieve the addresses of the pages that have been written to since the region has been allocated or the write-tracking state has been reset”.

  This feature can be used to track debuggers that may modify memory pages outside the expected pattern.

- **YARA Rule**: See the YARA rule for this technique in the subsection A.1.28
- **Results**: This YARA rule detected 319/997 malware files.
- **References**:
  - https://anti-debug.checkpoint.com/techniques/misc.html

4.29. Technique 29: Call to Interrupt Procedure

- **Description:** This anti-debugging technique involves using the INT n instruction to generate a call to the interrupt or exception handler specified with the destination operand.

  To implement this technique, the int 0x03 instruction is executed, followed by a ret (0xCD03, 0xC3) nested in a __try, __except block. If a debugger is present, the except block will not be executed, and the function will return TRUE, indicating that a debugger is running.

  This technique can be used to prevent analysts from analysing and manipulating the malware's code during runtime.

- **YARA Rule**: See the YARA rule for this technique in the subsection A.1.29
- **Results**: This YARA rule detected 2/997 malware files.
- **References**:
    - https://unprotect.it/technique/call-to-interrupt-procedure/

4.30. Technique 30: DbgPrint()

- **Description:** The debug functions such as ntdll!DbgPrint() and kernel32!OutputDebugStringW() cause the exception DBG_PRINTEXCEPTION_C (0x40010006). If a program is executed with an attached debugger, then the debugger will handle this exception. But if no debugger is present, and an exception handler is registered, this exception will be caught by the exception handler.
- **YARA Rule**: See the YARA rule for this technique in the subsection A.1.30
- **Results**: This YARA rule detected 22/997 malware files.
- **References**:
    - https://anti-debug.checkpoint.com/techniques/misc.html

4.31. Technique 31: FuncIn

- **Description:** FuncIn involves a payload staging strategy wherein the entire set of malicious functionalities is not contained within the malware file itself or any third-party file/network location (e.g., a web server). Instead, these functionalities are transmitted over the network by the Command and Control (C2) server when required.

  This approach addresses three primary issues in malware development. Firstly, it mitigates the size of the malware, as most functionalities are not directly embedded in the malware. Functioning as a network loader, the malware can be very compact (often just a few kilobytes). Secondly, it tackles the challenges posed by malware analysis and detection. The absence of most malicious functionalities physically within the malware or its immediate dependencies can make reverse engineering and detection more challenging. For instance, if the malware loader expects specific characteristics to be met before transmitting the rest of the payload (e.g., geographic location, system requirements, certain user or host machine behaviour, and properties), analysis becomes difficult until the complete payload is downloaded.

  Lastly, it addresses maintenance concerns. The majority of the code residing on the C2 side makes maintenance more convenient. Malicious actors do not need to update the remote loader but only the shellcodes (functionalities) located on the C2, transmitted when needed.
- **YARA Rule**: See the YARA rule for this technique in the subsection A.1.31
- **Results**: This YARA rule detected 734/997 malware files.
- **References**:
    - https://unprotect.it/technique/funcin/

4.32. Technique 32: RtlQueryProcessHeapInformation

- **Description:** the ntdll!RtlQueryProcessHeapInformation() function can be used to read the heap flags from the process memory of the current process.
- **YARA Rule**: See the YARA rule for this technique in the subsection A.1.32
- **Results**: This YARA rule detected 98/997 malware files.
- **References**:
    - https://anti-debug.checkpoint.com/techniques/debug-flags.html#using-win32-api-ntqueryinformationprocess

4.33. Technique 33: PEB!BeingDebugged Flag

- **Description:** This method is just another way to check BeingDebugged flag of PEB without calling IsDebuggerPresent().
- **YARA Rule**: See the YARA rule for this technique in the subsection A.1.33
- **Results**: This YARA rule detected 12/997 malware files.
- **References**:
    - https://anti-debug.checkpoint.com/techniques/debug-flags.html#manual-checks-peb-beingdebugged-flag

4.34. Technique 34: Heap Protection

- **Description:** If the HEAP_TAIL_CHECKING_ENABLED flag is set in NtGlobalFlag, the sequence 0xABABABAB will be appended (twice in 32-Bit and 4 times in 64-Bit Windows) at the end of the allocated heap block.
- If the HEAP_FREE_CHECKING_ENABLED flag is set in NtGlobalFlag, the sequence 0xFEEEFEEE will be appended if additional bytes are required to fill in the empty space until the next memory block.
- **YARA Rule**: See the YARA rule for this technique in the subsection A.1.34
- **Results**: This YARA rule detected 12/997 malware files.
- **References**:
    - https://anti-debug.checkpoint.com/techniques/debug-flags.html#manual-checks-heap-protection

4.35. Technique 35: SwitchDesktop()

- **Description:** Windows supports multiple desktops per session. It is possible to select a different active desktop, which has the effect of hiding the windows of the previously active desktop, and with no obvious way to switch back to the old desktop.

  Further, the mouse and keyboard events from the debugged process desktop will no longer be delivered to the debugger, because their source is no longer shared. This obviously makes debugging impossible.
- **YARA Rule**: See the YARA rule for this technique in the subsection A.1.35

- **Results**: This YARA rule detected 102/997 malware files.
- **References**:
  - https://anti-debug.checkpoint.com/techniques/misc.html

4.36. Technique 36: OpenProcess()

- **Description:** Some debuggers can be detected by using the kernel32!OpenProcess() function on the csrss.exe process. The call will succeed only if the user for the process is a member of the administrators group and has debug privileges.
- **YARA Rule**: See the YARA rule for this technique in the subsection A.1.36
- **Results**: This YARA rule detected 1/997 malware files.
- **References**:
  - https://anti-debug.checkpoint.com/techniques/object-handles.html#openprocess

4.37. Technique 37: CreateFile()

- **Description:** When the CREATE_PROCESS_DEBUG_EVENT event occurs, the handle of the debugged file is stored in the CREATE_PROCESS_DEBUG_INFO structure. Therefore, debuggers can read the debug information from this file. If this handle is not closed by the debugger, the file won't be opened with exclusive access. Some debuggers can forget to close the handle.

  This trick uses kernel32!CreateFileW() (or kernel32!CreateFileA()) to exclusively open the file of the current process. If the call fails, we can consider that the current process is being run in the presence of a debugger.
- **YARA Rule**: See the YARA rule for this technique in the subsection A.1.37
- **Results**: This YARA rule detected 1/997 malware files.
- **References**:
  - https://anti-debug.checkpoint.com/techniques/object-handles.html#createfile

4.38. Technique 38: LoadLibrary()

- **Description:** When a file is loaded to process memory using the kernel32!LoadLibraryW() (or kernel32!LoadLibraryA()) function, the LOAD_DLL_DEBUG_EVENT event occurs. The handle of the loaded file will be stored in the LOAD_DLL_DEBUG_INFO structure. Therefore, debuggers can read the debug information from this file. If this handle is not closed by the debugger, the file won't be opened with exclusive access. Some debuggers can forget to close the handle.

  To check for the debugger presence, we can load any file using kernel32!LoadLibraryA() and try to exclusively open it using kernel32!CreateFileA(). If the kernel32!CreateFileA() call fails, it indicates that the debugger is present.

- **YARA Rule**: See the YARA rule for this technique in the subsection A.1.38
- **Results**: This YARA rule detected 564/997 malware files.
- **References**:
    - https://anti-debug.checkpoint.com/techniques/object-handles.html#loadlibrary

4.39. Technique 39: RaiseException()

- **Description**: Exceptions such as DBC_CONTROL_C or DBG_RIPEVENT are not passed to exception handlers of the current process and are consumed by a debugger. This lets us register an exception handler, raise these exceptions using the kernel32!RaiseException() function, and check whether the control is passed to our handler. If the exception handler is not called, the process is likely under debugging.
- **YARA Rule**: See the YARA rule for this technique in the subsection A.1.39
- **Results**: This YARA rule detected 291/997 malware files.
- **References**:
    - https://anti-debug.checkpoint.com/techniques/exceptions.html#raiseexception

4.40. Technique 40: Hiding Control Flow with Exception Handlers

- **Description:** This approach does not check whether a debugger is present, but it helps to hide the control flow of the program in the sequence of exception handlers.

  We can register an exception handler (structured or vectored) which raises another exception which is passed to the next handler which raises the next exception, and so on. Finally, the sequence of handlers should lead to the procedure that we wanted to hide.
- **YARA Rule**: See the YARA rule for this technique in the subsection A.1.40
- **Results**: This YARA rule detected 291/997 malware files.
- **References**:
    - https://anti-debug.checkpoint.com/techniques/exceptions.html

4.41. Technique 41: ZwGetTickCount() / KiGetTickCount()

- **Description:** Both functions are used only from Kernel Mode.
- Just like User Mode GetTickCount() or GetSystemTime(), Kernel Mode ZwGetTickCount() reads from the KUSER_SHARED_DATA page. This page is mapped read-only into the user mode range of the virtual address and read-write in the kernel range. The system clock tick updates the system time, which is stored directly in this page.

- ZwGetTickCount() is used the same way as GetTickCount(). Using KiGetTickCount() is faster than calling ZwGetTickCount(), but slightly slower than reading from the KUSER_SHARED_DATA page directly.
- **YARA Rule**: See the YARA rule for this technique in the subsection A.1.41
- **Results**: This YARA rule detected 47/997 malware files.
- **References**:
  - https://anti-debug.checkpoint.com/techniques/timing.html

4.42. Technique 42: BlockInput()

- **Description:** The function user32!BlockInput() can block all mouse and keyboard events, which is quite an effective way to disable a debugger. On Windows Vista and higher versions, this call requires administrator privileges.

  We can also detect whether a tool that hooks the user32!BlockInput() and other anti-debug calls is present. The function allows the input to be blocked only once. The second call will return FALSE. If the function returns TRUE regardless of the input, it may indicate that some hooking solution is present.
- **YARA Rule**: See the YARA rule for this technique in the subsection A.1.42
- **Results**: This YARA rule detected 68/997 malware files.
- **References**:
  - https://anti-debug.checkpoint.com/techniques/interactive.html

4.43. Technique 43: Selectors

- **Description:** Selector values might appear to be stable, but they are actually volatile in certain circumstances, and also depending on the version of Windows. For example, a selector value can be set within a thread, but it might not hold that value for very long. Certain events might cause the selector value to be changed back to its default value. One such event is an exception. In the context of a debugger, the single-step exception is still an exception, which can cause some unexpected behaviour.
- **YARA Rule**: See the YARA rule for this technique in the subsection A.1.43
- **Results**: This YARA rule detected 340/997 malware files.
- **References**:
  - https://anti-debug.checkpoint.com/techniques/misc.html

4.44. Technique 44: Stack segment register

- **Description:** This is a trick that can be used to detect if the program is being traced. The trick consists of tracing over the following sequence of assembly instructions:

```
push ss
pop ss
```

pushf

After single-stepping in a debugger through this code, the Trap Flag will be set. Usually it's not visible as debuggers clear the Trap Flag after each debugger event is delivered. However, if we previously save EFLAGS to the stack, we'll be able to check whether the Trap Flag is set.

- **YARA Rule**: See the YARA rule for this technique in the subsection A.1.44
- **Results**: This YARA rule detected 16/997 malware files.
- **References**:
  - https://anti-debug.checkpoint.com/techniques/assembly.html

4.45. Technique 45: Instruction Prefixes

- **Description:** This trick works only in some debuggers. It abuses the way how these debuggers handle instruction prefixes.

  If we execute the following code in OllyDbg, after stepping to the first byte F3, we'll immediately get to the end of try block. The debugger just skips the prefix and gives the control to the INT1 instruction.

  If we run the same code without a debugger, an exception will be raised and we'll get to except block.

- **YARA Rule**: See the YARA rule for this technique in the subsection A.1.45
- **Results**: This YARA rule detected 20/997 malware files.
- **References**:
  - https://anti-debug.checkpoint.com/techniques/misc.html

4.46. Technique 46: Self-Debugging

- **Description:** There are at least three functions that can be used to attach as a debugger to a running process:
  - kernel32!DebugActiveProcess()
  - ntdll!DbgUiDebugActiveProcess()
  - ntdll!NtDebugActiveProcess()

  As only one debugger can be attached to a process at a time, a failure to attach to the process might indicate the presence of another debugger.

  In the example below, we run the second instance of our process which tries to attach a debugger to its parent (the first instance of the process). If kenel32!DebugActiveProcess() finishes unsuccessfully, we set the named event which was created by the first instance. If the event is set, the first instance understands that a debugger is present.

- **YARA Rule**: See the YARA rule for this technique in the subsection A.1.46
- **Results**: This YARA rule detected 322/997 malware files.
- **References**:

- https://anti-debug.checkpoint.com/techniques/interactive.html#self-debugging

4.47. Technique 47: GenerateConsoleCtrlEvent()

- **Description:** When a user presses Ctrl+C or Ctrl+Break and a console window is in the focus, Windows checks if there is a handler for this event. All console processes have a default handler function that calls the kernel32!ExitProcess() function. However, we can register a custom handler for these events which neglects the Ctrl+C or Ctrl+Break signals.

  However, if a console process is being debugged and CTRL+C signals have not been disabled, the system generates a DBG_CONTROL_C exception. Usually this exception is intercepted by a debugger, but if we register an exception handler, we will be able to check whether DBG_CONTROL_C is raised. If we intercepted the DBG_CONTROL_C exception in our own exception handler, it may indicate that the process is being debugged.

- **YARA Rule**: See the YARA rule for this technique in the subsection A.1.47
- **Results**: This YARA rule detected 49/997 malware files.
- **References**:
  - https://anti-debug.checkpoint.com/techniques/interactive.html

5. Budget

In this section, I will detail the budget considerations for my project, focusing on three primary areas: electricity costs, peripheral expenses, and the hours dedicated to the work.

Electricity costs are calculated based on the power consumption of my equipment and the duration of use. Peripheral expenses include the cost of necessary hardware. Lastly, the hours I have invested in this project will be quantified, reflecting the time and effort dedicated to research, development, and documentation.

Hardware	Price
Tower computer	600 €
Monitor ASUS	140 €
Monitor BENQ	120 €
Keyboard	25 €
Webcam	35 €
Mouse	25 €
Total cost	**945 €**

Table 3: Hardware used. Sources: Own elaboration

To calculate the budget of the hours dedicated to work I contemplated all the roles I simulated doing during the project, those are: Researcher, Analyst and Programmer.

To know the average wage of every job I did the research on Glasdoor [17], I contemplated myself as Junior in all the roles due to my lack of professional experience.

Role	Average wage	Hours	Total cost
Analyst Junior	11.8 €/h	40	472 €
Researcher Junior	11.6 €/h	140	1624 €
Programmer junior	10.2 €/h	115	1173 €
Total cost			**2096 €**

Table 4: Hours and average wage of every Role. Sources: Own elaboration

The electricity cost of the project is simple to compute, just multiply the electricity cost by the time dedicated.

Considering the peripheral I estimated a consumption of 0.262 kW per hour and the average €/KWh is 0.1365, this data was obtained via Tarifaluzhora [18], so the electricity consumption will be:

kW/h	Hours	€/kWh	Total cost
0.262	295	0.1365	**10.55 €**

Table 5. Electricity cost. Sources: Own elaboration

Finally, to compute the total cost of the project I'm going to add the electricity costs, peripheral expenses, and the hours dedicated and add a 10% to compute the contingencies

Type of cost	Cost
Electricity	10.55 €
Personal	2096 €
Peripherals	945 €
Contingencies	305.15 €
Total cost	**3356.7 €**

Table 6: Project total cost. Sources: Own elaboration

The total project cost amounts to 3356 €. This includes all expenses and anticipated expenditures. The budget allocation ensures all project requirements are met efficiently.

6. Environment Impact

This section describes the potential environmental impacts, both negative and positive, resulting from the tasks involved in completing this thesis and the outcomes achieved.

6.1. Negative impact

Energy Consumption:

- During the research and development of this thesis, computer hardware was used to run and test YARA rules against malware sample databases. This process involved energy consumption, in this case electricity.

Electronic Waste:

- The intensive use of electronic devices, such as computers and servers, can accelerate their wear and tear, bringing them closer to the end of their usable life. As these devices age, they may require more frequent maintenance or replacement, which can lead to an increase in electronic waste. Proper disposal and recycling of these devices are crucial to minimize their environmental impact, as electronic waste contains hazardous materials that pose significant environmental risks.

6.2. Positive impact

Enhanced Cybersecurity:

- The primary objective of this research is to improve the detection of anti-debugging techniques used by malware. By developing and validating YARA rules, this project provides practical tools that enhance cybersecurity measures. Improved cybersecurity can indirectly benefit the environment by protecting critical infrastructure and reducing the risk of cyber-attacks that could lead to environmental damage. For example, protecting industrial control systems in energy plants from malware attacks can prevent potential environmental disasters.

Resource Efficiency:

- By sharing the developed YARA rules with the cybersecurity community, this project promotes collaboration and resource efficiency. Instead of multiple organizations duplicating efforts to develop similar detection mechanisms, sharing these rules allows for more efficient use of resources, reducing the overall environmental footprint associated with this type of research and development.

Reduction in Cybercrime-Related Resource Use:

- Effective malware detection reduces the frequency and severity of cyber-attacks, which often require significant resources to mitigate and recover from. By preventing these attacks, there is a reduction in the need for emergency responses, thereby saving energy and resources that would otherwise be expended in addressing cyber incidents.

Conclusion:

While the direct environmental impact of this thesis is minimal, the indirect effects, particularly the positive contributions to cybersecurity, can have broader environmental benefits. The research and tools developed through this project contribute to a more secure and efficient use of resources.

7. Conclusions and future development

This study has investigated anti-analysis techniques employed by malware developers to evade debugging, drawing from Cyber Threat Intelligence sources. The following points highlight the significant findings of this research:

Specific YARA rules have been developed to detect a wide range of anti-debugging techniques. Some YARA were created from scratch being quite experimental. These rules have proven effective in identifying malicious behaviours attempting to evade debugging environments. In this project a total amount of 47 YARA rules were tested. The main sources of information have been UnProtect Project [7] and Check Point Research [6] due to the quality and well-structured nature of the information.

The validation of YARA rules was conducted using a database of malware samples, ensuring their efficacy in real-world scenarios. However, since the experiments were conducted with a database full of malware without any benign code, the false positive rate could not be evaluated.

Other sources, like VirusTotal [5], primarily consider anti-debugging methods that exploit timers. Yet, as the results have shown, there are more valid approaches to detect malware.

The study underscores the potential utility of these techniques as effective methods for detecting and malware threats, thereby contributing to enhanced cybersecurity measures.

As for the future, the project is far from finished since more YARA rules can be created that exploit different ideas. In this same project, I had to discard some because they were beyond my understanding. This project could be extended to develop more YARA rules or to deepen and improve the ones already developed.

The project has been designed to be scalable, which is why the source code is available on my GitHub [19], ready to be used by any community member interested in the topic.

Bibliography

[1] G. Vila, «Detection of anti-analysis malware techniques: Anti-VM and Anti-Sandbox», TFG, Escola Tècnica Superior d'Enginyeria de Telecomunicació de Barcelona of the Universitat Politècnica de Catalunya, 2024.

[2] A.-T.-T. I. I.-S. Institute, «AV-ATLAS - Malware & PUA», AV-ATLAS - Malware & PUA. [En línea]. Disponible en: https://portal.av-atlas.org/malware

[3] «Zotero | Your personal research assistant». [En línea]. Disponible en: https://www.zotero.org/

[4] «Free Online Gantt Chart Software». [En línea]. Disponible en: https://www.onlinegantt.com

[5] «VirusTotal - Home». [En línea]. Disponible en: https://www.virustotal.com/gui/home/upload

[6] Research, Check Point, «Anti-Debug Tricks», Anti-Debug Tricks. [En línea]. Disponible en: https://anti-debug.checkpoint.com/

[7] T. Roccia, «Home - Unprotect Project». [En línea]. Disponible en: https://unprotect.it/

[8] @yararules, «rules/antidebug_antivm/antidebug_antivm.yar at master · Yara-Rules/rules», GitHub. [En línea]. Disponible en: https://github.com/Yara-Rules/rules/blob/master/antidebug_antivm/antidebug_antivm.yar

[9] M. Sikorski y A. Honig, *Practical Malware Analysis: The Hands-On Guide to Dissecting Malicious Software*. No Starch Press, Inc., 2012.

[10] VirusTotal, «Welcome to YARA's documentation! — yara 4.5.0 documentation». [En línea]. Disponible en: https://yara.readthedocs.io/en/latest/

[11] N. Arkaan, «CAPA for Triage Malware Analysis», MII Cyber Security Consulting Services. [En línea]. Disponible en: https://medium.com/mii-cybersec/capa-for-triage-malware-analysis-4f99431ff08f

[12] «mandiant/capa-rules». [En línea]. Disponible en: https://github.com/mandiant/capa-rules

[13] T. Roccia, «Yara Toolkit». [En línea]. Disponible en: https://yaratoolkit.securitybreak.io/

[14] CrowdStrike, «Free Automated Malware Analysis Service - powered by Falcon Sandbox». [En línea]. Disponible en: https://hybrid-analysis.com/

[15] «VirusShare.com». [En línea]. Disponible en: https://virusshare.com/

[16] Microsoft, «Visual Studio Code - Code Editing. Redefined». [En línea]. Disponible en: https://code.visualstudio.com/

[17] Glassdoor LLC, «Búsqueda de empleo en Glassdoor», Glassdoor. [En línea]. Disponible en: https://www.glassdoor.es/index.htm

[18] «Precio de la tarifa de luz por horas HOY». [En línea]. Disponible en: https://tarifaluzhora.es/

[19] T. Prats, «General · Teo-Prats/yara_rules», GitHub. [En línea]. Disponible en: https://github.com/Teo-Prats/yara_rules

Appendix

A.1. Anti-Debugging Techniques

Is important to consider that different rules may use the same solutions despite originating from different ideas leading to some overlapping's.

All the rules are listed in this section but can be found too in my public repository in GitHub [19] .

A.1.1. Technique 1

The following YARA rule will detect the malware if there is present the string IsDebuggerPresent.

```
rule Detect_IsDebuggerPresent : AntiDebug {
    meta:
        author = "naxonez"
        reference =
"https://github.com/naxonez/yaraRules/blob/master/AntiDebugging.yara"
    strings:
        $1 ="IsDebuggerPresent"
    condition:
         any of them
}
```

And here are the captures related to the testing:

After choosing randomly one of the hashes of the detected virus we search for it in VirusTotal trying to contrast the information.

As we see in the image below it is indeed a malware.

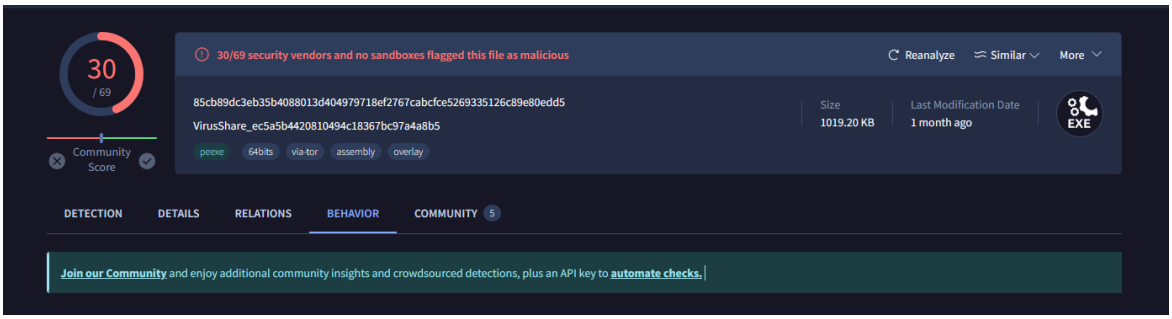

Figure 7: Technique 1 - Virus Total report 1

And in the next image, in the behaviour section, we can see in the capabilities of the malware which Anti-Analysis methods are registered that this malware used. However, it is not written to use a debug detector.

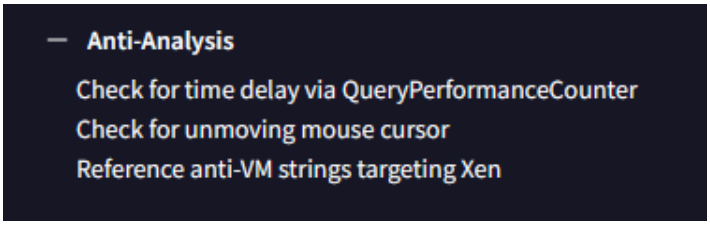

Figure 8: Technique 1 - Virus Total report 2

A.1.2. Technique 2

The following YARA rule will detect the malware if there is present the string NtQueryInformationProcess.

```
rule NtQueryInformationProcess: AntiDebug {
    meta:
        description = "Detect NtQueryInformationProcess as anti-debug"
        author = "Teo-Prats"
        comment = "Modified rule from UnProtect"
        reference =
"https://github.com/naxonez/yaraRules/blob/master/AntiDebugging.yara"
    strings:
        $1 = "NtQueryInformationProcess"
    condition:
       any of them
}
```

After choosing randomly one of the hashes of the detected virus we search for it in VirusTotal trying to contrast the information.

As we see in the image below it is indeed a malware.

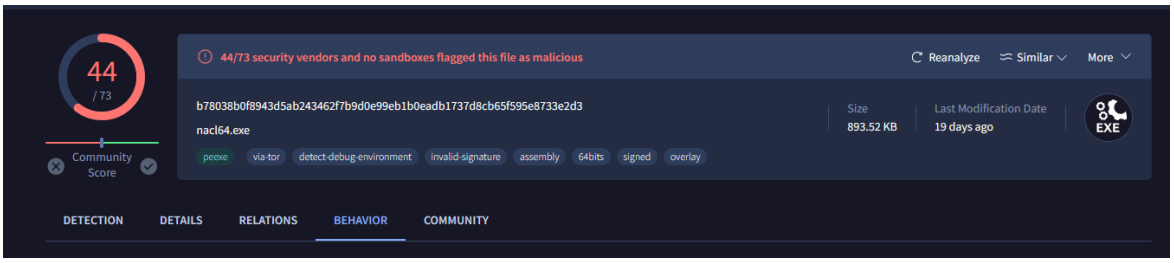

Figure 9: Technique 2 - Virus Total report 1

And in the next image, in the behaviour section, we can see in the capabilities of the malware which Anti-Analysis methods are registered that this malware used. However, they have only listed the QueryPerformanceCounter.

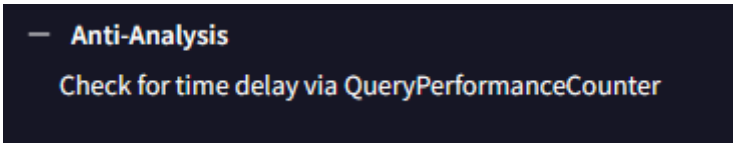

Figure 10: Technique 2 - Virus Total report 2

A.1.3. Technique 3

The following YARA rule will detect the malware if there is present the string NtSetInformationThread.

```
rule NtSetInformationThread: AntiDebug {
    meta:
        description = "Detect NtSetInformationThread as anti-debug"
        author = "Teo-Prats"
        comment = "Modified rule from UnProtect"
        reference =
"https://unprotect.it/technique/ntsetinformationthread/"
    strings:
        $1 = "NtSetInformationThread"
    condition:
       any of them
}
```

After choosing randomly one of the hashes between all the detected virus we search for it in VirusTotal trying to contrast the information.

As we see in the image below it is indeed a malware.

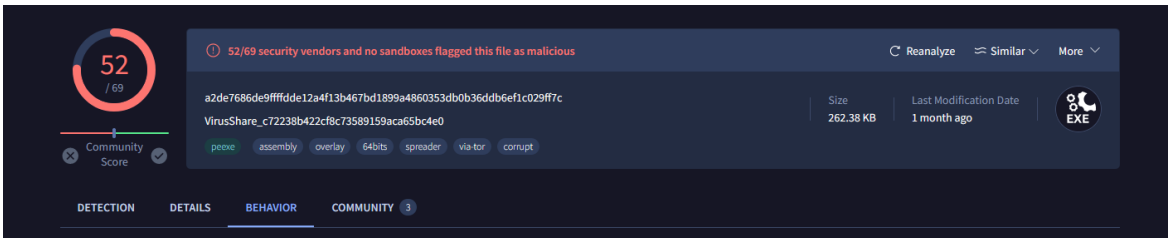

Figure 11: Technique 3 - Virus Total report 1

And in the next image, in the behaviour section, we can see in the capabilities of the malware which Anti-Analysis methods are registered that this malware used. However, they do not have any Anti-analysis method so we can say that our rule has been more efficient.

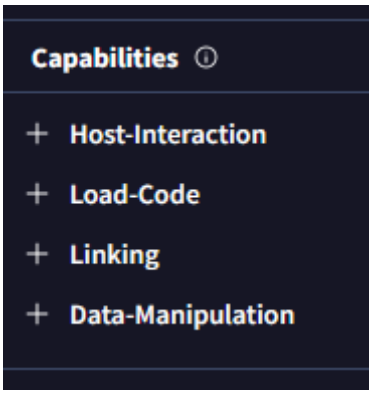

Figure 12: Technique 3 - Virus Total report 2

A.1.4. Technique 4

The following YARA rule will detect the malware if there is present the string NtQueryObject in 8bits (ASCII) or 16bits (UTF-16).

```
rule NtQueryObject: AntiDebug {
    meta:
        description = "Detect NtQueryObject as anti-debug"
        author = "Teo-Prats"
        comment = "Modified rule from UnProtect"
        reference = "https://unprotect.it/technique/ntqueryobject/"
    strings:
        $1 = "NtQueryObject" wide ascii
    condition:
       any of them
}
```

After choosing randomly one of the hashes between all the detected virus we search for it in VirusTotal trying to contrast the information.

As we see in the image below it is indeed a malware

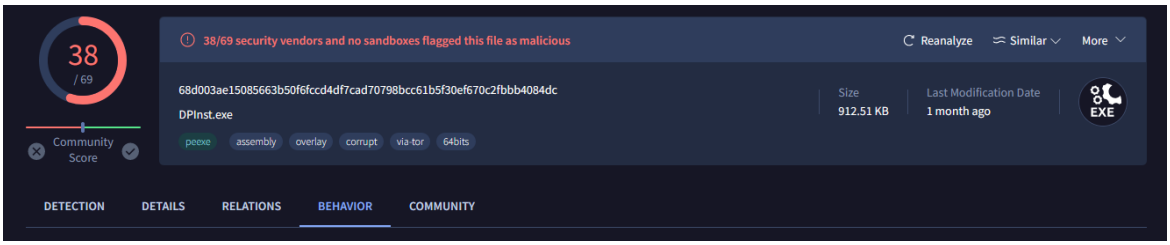

Figure 13: Technique 4 - Virus Total report 1

And in the next image, in the behaviour section, we can see in the capabilities of the malware which Anti-Analysis methods are registered that this malware used. However, they do not have any Anti-analysis method so we can say that our rule has been more efficient.

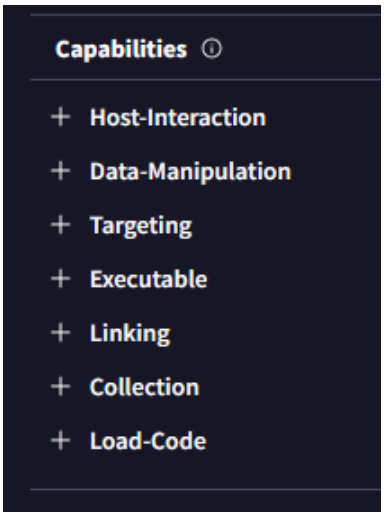

Figure 14: Technique 4 - Virus Total report 2

A.1.5. Technique 5

The following YARA rule will detect the malware if the malware imports either the OutputDebugStringW (wide/UTF-16 version) or OutputDebugStringA (ASCII version) function from the kernel32.dll library.

```
import "pe"
rule OutputDebugString: AntiDebug
{
    meta:
        Author = "Teo-Prats"
        Description = "Detect OutputDebugstring"
        Reference = "http://twitter.com/j0sm1"

    condition:
        pe.imports("kernel32.dll","OutputDebugStringW")
        or
        pe.imports("kernel32.dll","OutputDebugStringA")
}
```

After choosing randomly one of the hashes between all the detected virus we search for it in VirusTotal trying to contrast the information.

As we see in the image below it is indeed a malware

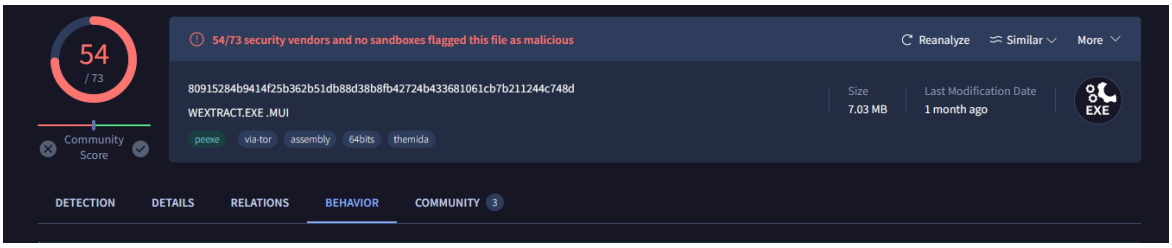

Figure 15: Technique 5 - Virus Total report 1

And in the next image, in the behaviour section, we can see in the capabilities of the malware which Anti-Analysis methods are registered that this malware used. However, they have only anti-VM methods.

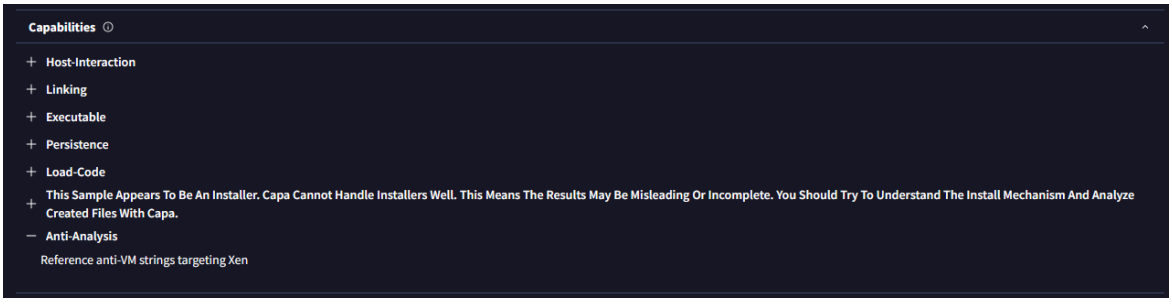

Figure 16: Technique 5 - Virus Total report 2

A.1.6. Technique 6

The YARA rule will detect the malware if the YARA detects one of the following strings: EventPairHandles, RtlCreateQueryDebugBuffer, RtlQueryProcessHeapInformation.

```
rule EventPairHandles: AntiDebug {
    meta:
        description = "Detect EventPairHandlesas anti-debug"
        author = "Teo-Prats"
        comment = "Modified rule from UnProtect"
        Reference = "https://unprotect.it/technique/eventpairhandles/"
    strings:
        $1 = "EventPairHandles"
        $2 = "RtlCreateQueryDebugBuffer"
        $3 = "RtlQueryProcessHeapInformation"
    condition:
        1 of them
}
```

After choosing randomly one of the hashes between all the detected virus we search for it in VirusTotal trying to contrast the information.

As we see in the image below it is indeed a malware

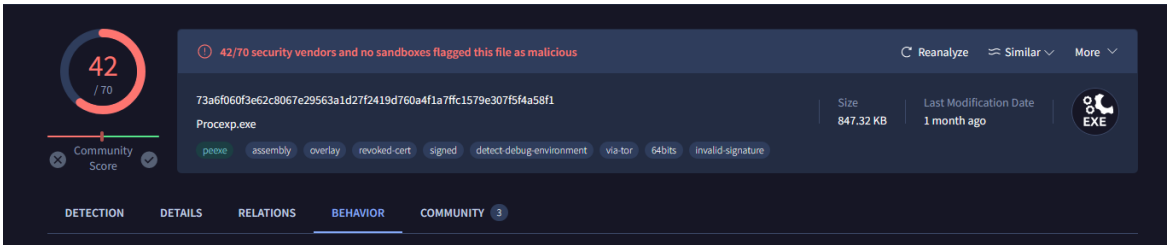

Figure 17: Technique 6 - Virus Total report 1

And in the next image, in the behaviour section, we can see in the capabilities of the malware which Anti-Analysis methods are registered that this malware used. However, they have listed the use of analysis tools strings but not the ones that I used.

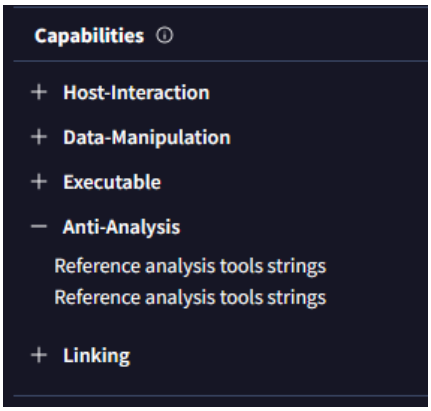

Figure 18: Technique 6 - Virus Total report 2

A.1.7. Technique 7

The YARA rule will detect the malware if the YARA detects one of the following strings: CsrGetProcessID, GetModuleHandle.

```
rule CsrGetProcessID: AntiDebug {
    meta:
        description = "Detect CsrGetProcessID as anti-debug"
        author = "Teo-Prats"
        comment = "Modified rule from UnProtect"
         Reference = "https://unprotect.it/technique/csrgetprocessid/"
    strings:
        $1 = "CsrGetProcessID" fullword
        $2 = "GetModuleHandle" fullword
    condition:
        1 of them
}
```

As this malware did not detect any malware sample, no testing has been performed.

A.1.8. Technique 8

The YARA rule will detect the malware if the YARA detects one of the following strings: NtClose, CloseHandle.

```
rule CloseHandle: AntiDebug {
    meta:
        description = "Detect CloseHandle as anti-debug"
        author = "Teo-Prats"
        comment = "Modified rule from UnProtect"
        Reference = "https://unprotect.it/technique/closehandle-ntclose/"
    strings:
        $1 = "NtClose" fullword
        $2 = "CloseHandle" fullword
    condition:
        any of them
}
```

After choosing randomly one of the hashes between all the detected virus we search for it in VirusTotal trying to contrast the information.

As we see in the image below it is indeed a malware despite of only 2 vendors marked it as malicious.

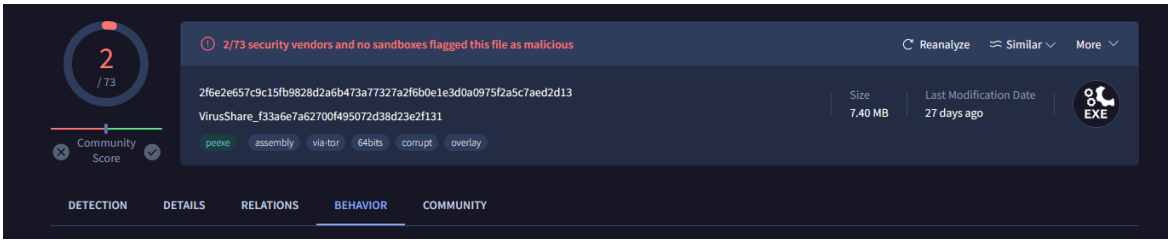

Figure 19: Technique 8 - Virus Total report 1

And in the next image, in the behaviour section, we can see in the capabilities of the malware which Anti-Analysis methods are registered that this malware used.

The rules I am creating detect malware using unlisted methods, as VirusTotal reports the virus uses anti-VM techniques, but I have identified it by searching for specific strings.

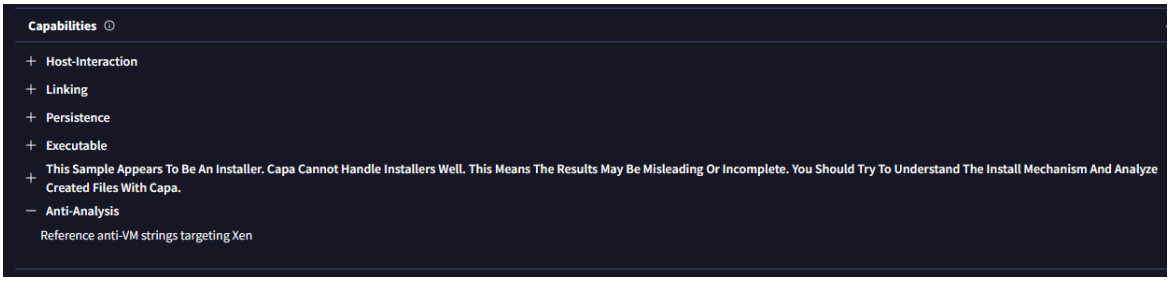

Figure 20: Technique 8 - Virus Total report 2

A.1.9. Technique 9

The YARA rule will detect the malware if the YARA detects one of the following strings: NtGlobalFlags

```
rule DebuggerCheck__GlobalFlags {
    meta:
    description = "Rule to detect NtGlobalFlags debugger check"
        author = "Thibault Seret"
        date = "2020-09-26"
        Reference = "https://unprotect.it/technique/ntglobalflag/"
    strings:
        $s1 = "NtGlobalFlags"
    condition:
        any of them
}
```

As this malware did not detect any malware sample, no testing has been performed.

A.1.10. Technique 10

The YARA rule will detect the malware if the YARA detects the hexadecimal string 0F 31 which is the opcode of the instruction RDTSC.

```
rule RDTSC: AntiDebug {
    meta:
        description = "Detect RDTSC as anti-debug"
        author = "Teo-Prats"
        comment = "Modified rule from UnProtect"
        reference = "https://unprotect.it/technique/rdtsc/"
    strings:
        $1 = { 0F 31 }
    condition:
        $1
}
```

After choosing randomly one of the hashes between all the detected virus we search for it in VirusTotal trying to contrast the information

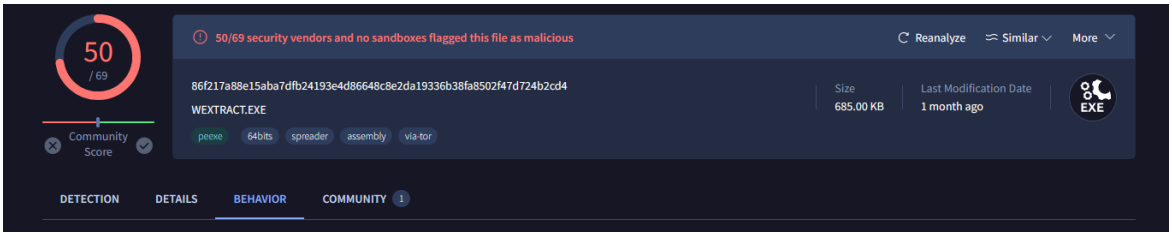

Figure 21: Technique 10 - Virus Total report 1

And in the next image, in the behaviour section, we can see in the capabilities of the malware which Anti-Analysis methods are registered that this malware used.

The rules I am creating detect malware using unlisted methods, as VirusTotal reports the virus uses anti-VM techniques, but I have identified it by searching for specific strings.

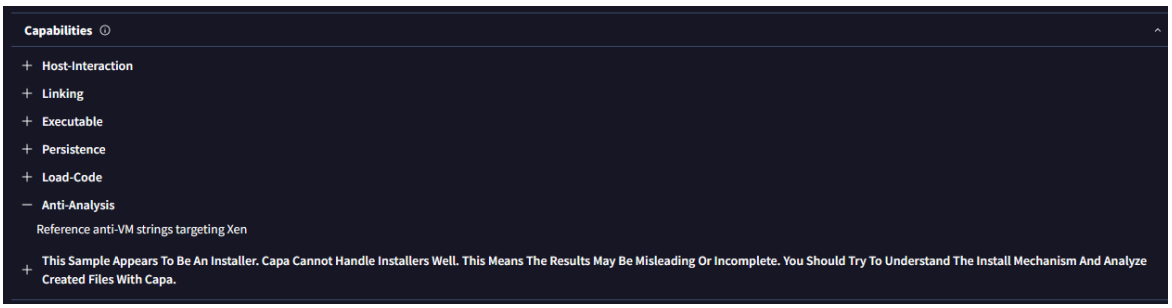

Figure 22: Technique 10 - Virus Total report 2

A.1.11. Technique 11

This YARA rule aims to identify files that meet the following criteria:

- First, the PE file imports the `FindWindowA` function from `user32.dll`
- The file contains a reference to "OLLYDBG" or "WinDbgFrameClass" which are debuggers.

```
import "pe"
rule Detect_FindWindowA_iat {
    meta:
        Author = "http://twitter.com/j0sm1"
        Description = "it's checked if FindWindowA() is imported"
        Date = "20/04/2015"
        Reference = "http://www.codeproject.com/Articles/30815/An-Anti-
Reverse-Engineering-Guide#OllyFindWindow"
    strings:
        $ollydbg = "OLLYDBG"
        $windbg = "WinDbgFrameClass"
    condition:
        pe.imports("user32.dll","FindWindowA") and ($ollydbg or $windbg)
}
```

As this malware did not detect any malware sample, no testing has been performed.

A.1.12. Technique 12

The YARA rule will detect the malware if the YARA detects the strings related to EnumProcess in 8bits (ASCII) or 16bits (UTF-16).

```
rule EnumProcess: AntiDebug {
    meta:
        description = "Detect EnumProcessas anti-debug"
         author = "Teo-Prats"
        comment = "Modified rule from UnProtect"
        reference = "https://unprotect.it/technique/detecting-running-
process-enumprocess-api/"
    strings:
        $1 = "EnumProcessModulesEx" wide ascii fullword
        $2 = "EnumProcesses" wide ascii fullword
        $3 = "EnumProcessModules" wide ascii fullword
    condition:
        any of them
}
```

After choosing randomly one of the hashes between all the detected virus we search for it in VirusTotal trying to contrast the information

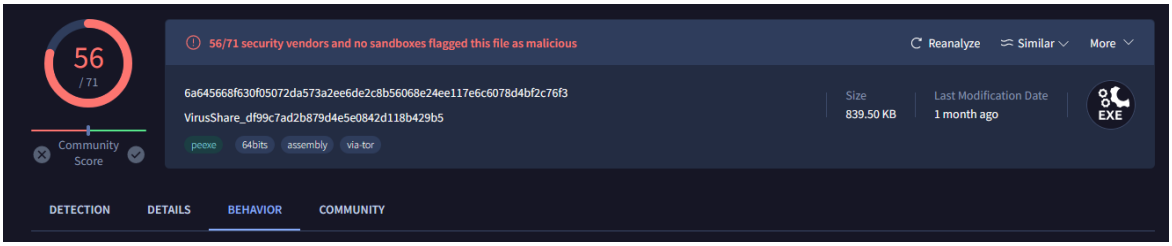

Figure 23: Technique 12 - Virus Total report 1

And in the next image, in the behaviour section, we can see in the capabilities of the malware which Anti-Analysis methods are registered that this malware used.

It is listed that the malware uses anti-debugging instructions but probably has a different approach than this YARA rule.

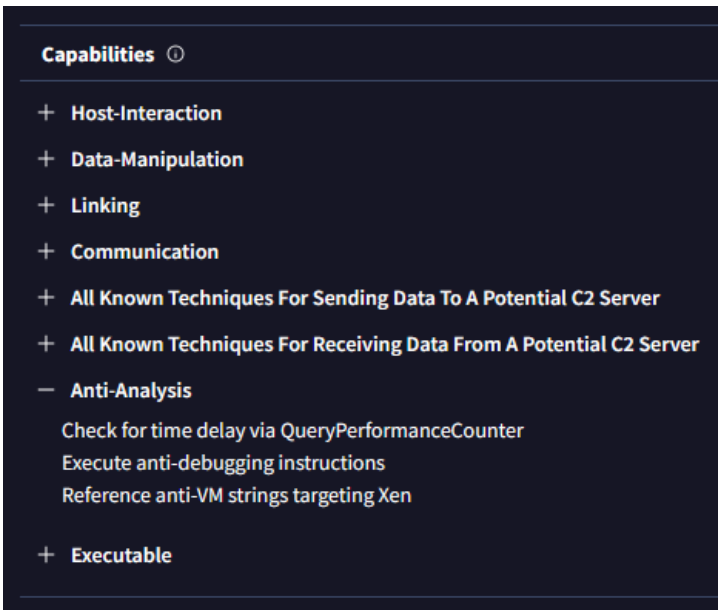

Figure 24: Technique 12 - Virus Total report 2

A.1.13. Technique 13

The YARA rule will detect the malware if the YARA detects the strings related to TLSCallback.

```
rule detect_tlscallback {
    meta:
        description = "Simple rule to detect tls callback as anti-debug."
        author = "Thomas Roccia | @fr0gger_"
    strings:
        $str1 = "TLS_CALLBACK" nocase
        $str2 = "TLScallback" nocase
    condition:
        any of them
}
```

After choosing randomly one of the hashes between all the detected virus we search for it in VirusTotal trying to contrast the information, only 1 vendor has classified this sample as malicious.

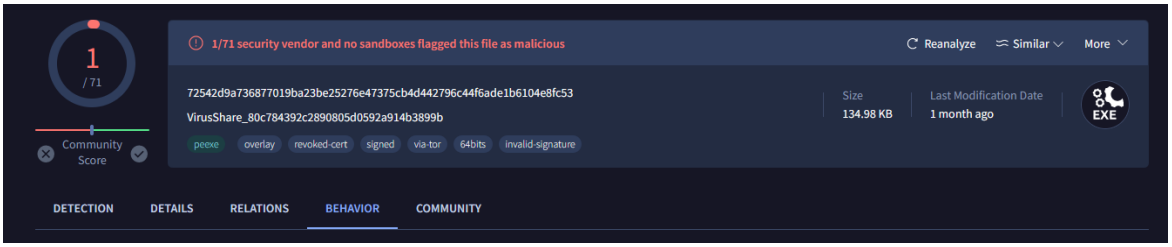

Figure 25: Technique 13 - Virus Total report 1

And again, in the capabilities section of the malware there is no listed any anti-debugging technique, only anti-VM.

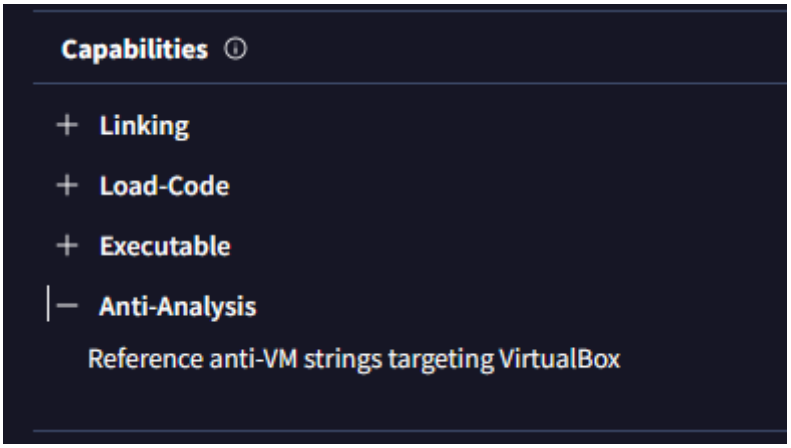

Figure 26: Technique 13 - Virus Total report 2

A.1.14. Technique 14

The YARA rule will detect the malware if the YARA finds the strings, which involve multiple %s, these format strings are used in anti-debugging techniques.

```
rule OllyDBG_BadFormatTrick: AntiDebug {
    meta:
        description = "Detect bad format not handled by Ollydbg"
        author = "Teo-Prats"
        comment = "Modified rule from UnProtect"
        reference = "https://unprotect.it/technique/bad-string-format/"
     strings:
        $str1 = "%s%s.exe" fullword ascii
        $str2 = "%s%s%s" fullword ascii
        $str3 = "%s%s%s%s" fullword ascii
        $str4 = "%s%s%s%s%s" fullword ascii
    condition:
        any of them
}
```

After choosing randomly one of the hashes between all the detected virus we search for it in VirusTotal trying to contrast the information.

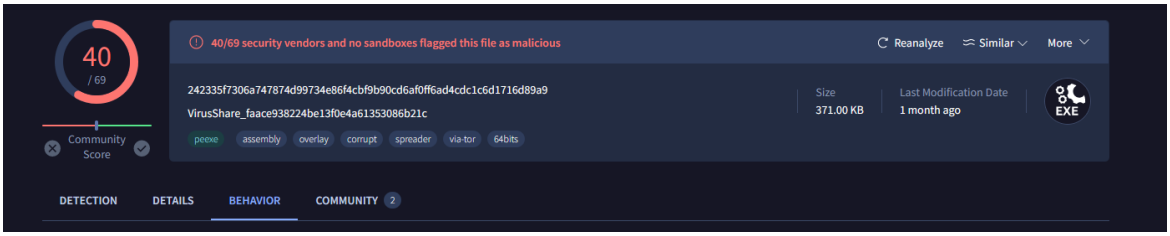

Figure 27: Technique 14 - Virus Total report 1

In the capabilities section is not listed any Anti-debugging method so it is proved again the effectiveness of this experimental rules.

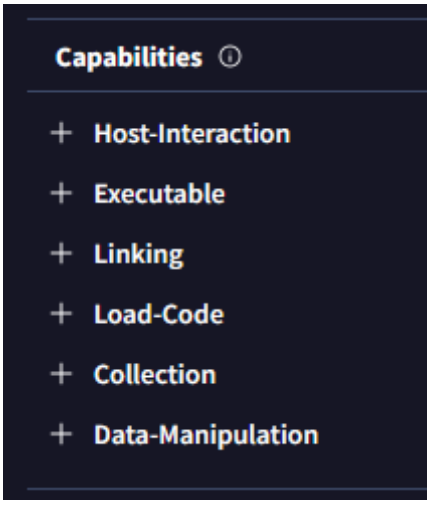

Figure 28: Technique 14 - Virus Total report 2

A.1.15. Technique 15

The YARA rule will detect the malware if the YARA finds the UnhandledExceptionFilter related strings.

```
rule SuspendThread: AntiDebug {
    meta:
        description = "Detect SuspendThread as anti-debug"
        author = "Unprotect"
        comment = "Experimental rule"
    strings:
        $1 = "UnhandledExcepFilter" fullword ascii
        $2 = "SetUnhandledExceptionFilter" fullword ascii
    condition:
       any of them
}
```

After choosing randomly one of the hashes between all the detected virus we search for it in VirusTotal trying to contrast the information.

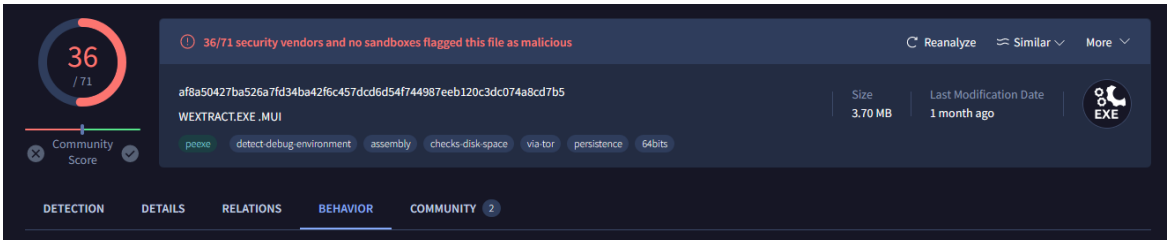

Figure 29: Technique 15 - Virus Total report 1

In the capabilities section is not listed any Anti-debugging method so it is proved again the effectiveness of this experimental rules.

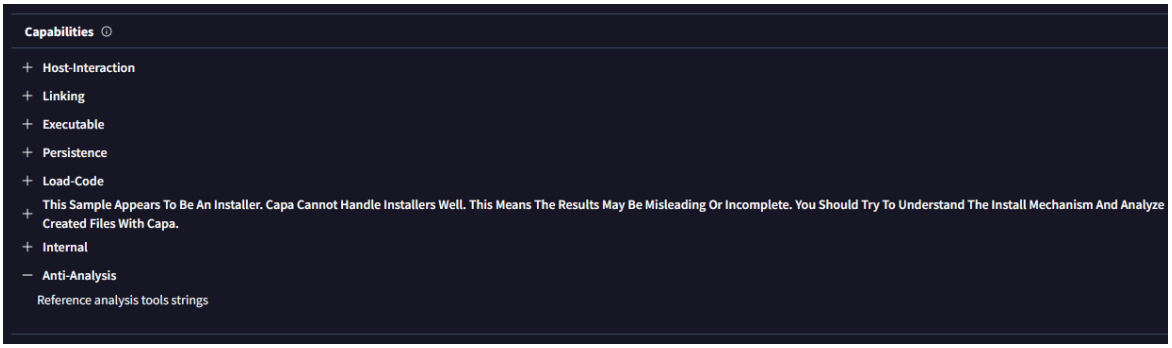

Figure 30: Technique 15 - Virus Total report 2

A.1.16. Technique 16

The YARA rule will detect the malware if the YARA finds the hexadecimal string CC and OF 0B which are the opcodes of the instructions INT3 and UD2.

```
rule ExceptionBased_AntiDebugging {
    meta:
        description = "Detects the use of INT 3 or UD2 instructions for
exception-based anti-debugging"
        author = "Teo-Prats"
        reference ="https://unprotect.it/technique/interrupts/"
    strings:
        $int3 = { CC }
        $ud2 = { 0F 0B }
    condition:
        $int3 or $ud2
}
```

After choosing randomly one of the hashes between all the detected virus we search for it in VirusTotal trying to contrast the information.

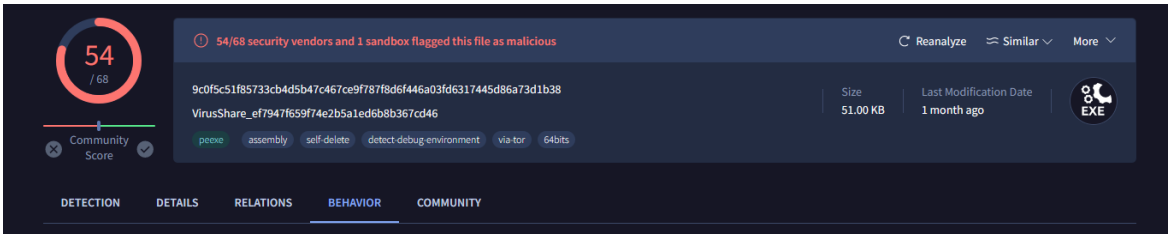

Figure 31: Technique 16 - Virus Total report 1

In the capabilities section is not listed any Anti-debugging method so it Is proved again the effectiveness of this experimental rules.

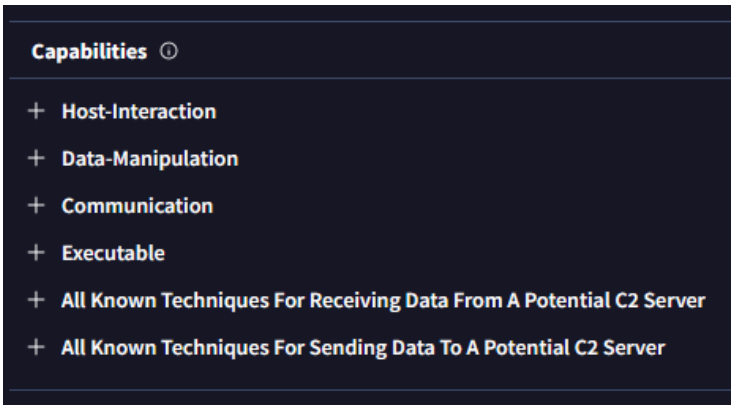

Figure 32: Technique 16 - Virus Total report 2

A.1.17. Technique 17

The YARA rule will detect the malware if the YARA finds the hexadecimal strings which are the opcodes of the instructions INT3, INTCD, INT03, INT2D, ICE.

```
rule Detect_Interrupt: AntiDebug {
    meta:
        description = "Detect Interrupt instruction"
        author = "Unprotect"
        comment = "Experimental rule / the rule can be slow to use"
        reference ="https://anti-debug.checkpoint.com/techniques/process-memory.html#breakpoints | https://unprotect.it/technique/int3-instruction-scanning/"
    strings:
        $int3 = { CC }
        $intCD = { CD }
        $int03 = { 03 }
        $int2D = { 2D }
        $ICE = { F1 }
    condition:
        any of them
}
```

After choosing randomly one of the hashes between all the detected virus we search for it in VirusTotal trying to contrast the information.

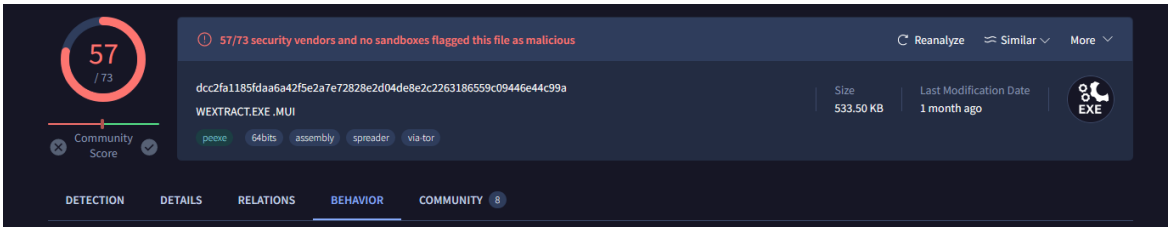

Figure 33: Technique 17 - Virus Total report 1

In the capabilities section is not listed any Anti-debugging method so it is proved again the effectiveness of this experimental rules.

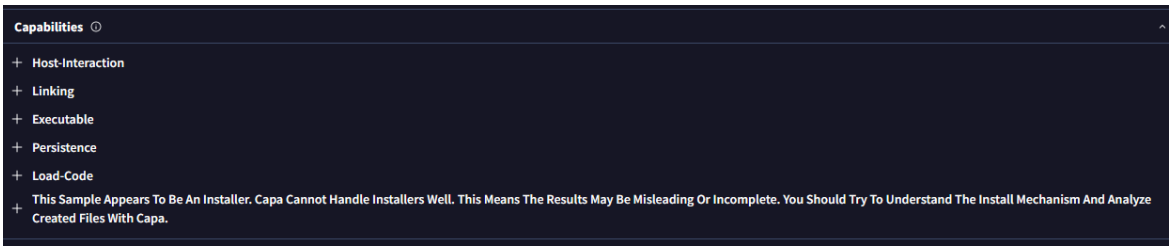

Figure 34: Technique 17 - Virus Total report 2

A.1.18. Technique 18

The YARA rule will detect the malware if the YARA finds the strings related to the SetDebugFilterState.

```
rule Detect_SetDebugFilterState: AntiDebug {
    meta:
        description = "Detect SetDebugFilterState as anti-debug"
        author = "Unprotect"
        comment = "Experimental rule"
    strings:
        $1 = "NtSetDebugFilterState" fullword ascii
        $2 = "DbgSetDebugFilterState" fullword ascii
    condition:
       any of them
}
```

As this malware did not detect any malware sample, no testing has been performed.

A.1.19. Technique 19

The YARA rule will detect the malware if the YARA finds the strings of specific API calls related to using guard pages.

```
rule GuardPages: AntiDebug {
    meta:
        description = "Detect Guard Pages as anti-debug"
        author = "Teo-Prats"
        comment = "Modified rule from UnProtect"
        Reference = "https://unprotect.it/technique/guard-pages/"
    strings:
        $1 = "GetSystemInfo" fullword ascii
        $2 = "VirtualAlloc" fullword ascii
        $3 = "RtlFillMemory" fullword ascii
        $4 ="VirtualProtect" fullword ascii
        $5 ="VirtualFree" fullword ascii
    condition:
        4 of them
}
```

After choosing randomly one of the hashes between all the detected virus we search for it in VirusTotal trying to contrast the information.

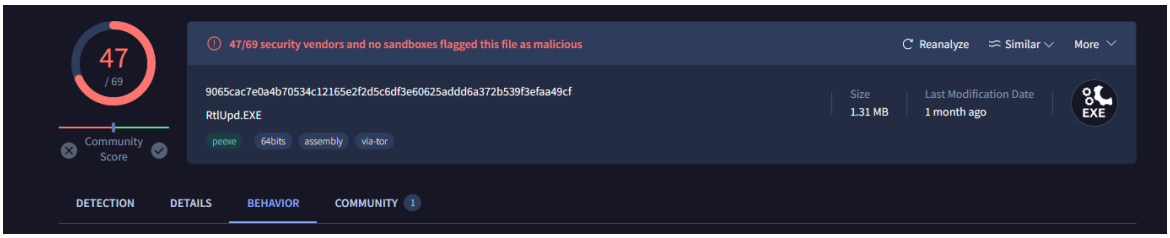

Figure 35: Technique 19 - Virus Total report 1

In the capabilities section is not listed any Anti-debugging method so it is proved again the effectiveness of this experimental rules.

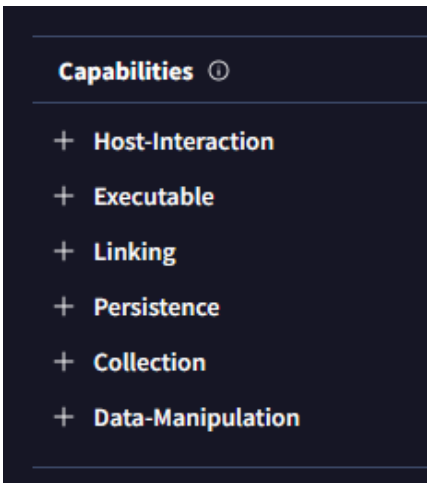

Figure 36: Technique 19 - Virus Total report 2

A.1.20. Technique 20

The YARA rule will detect the malware if the YARA finds the strings typically used in thread manipulation.

```
rule SuspendThread: AntiDebug {
    meta:
        description = "Detect SuspendThread as anti-debug"
        author = "Teo-Prats"
        comment = "Modified rule from UnProtect"
        Reference = "https://unprotect.it/technique/suspendthread/"
    strings:
        $1 = "SuspendThread" fullword ascii
        $2 = "NtSuspendThread" fullword ascii
        $3 = "OpenThread" fullword ascii
        $4 ="SetThreadContext" fullword ascii
        $5 ="SetInformationThread" fullword ascii
        $x1 ="CreateToolHelp32Snapshot" fullword ascii
        $x2 ="EnumWindows" fullword ascii
    condition:
        any of ($x1, $x2) and 2 of ($1, $2, $3, $4, $5)
}
```

After choosing randomly one of the hashes between all the detected virus we search for it in VirusTotal trying to contrast the information.

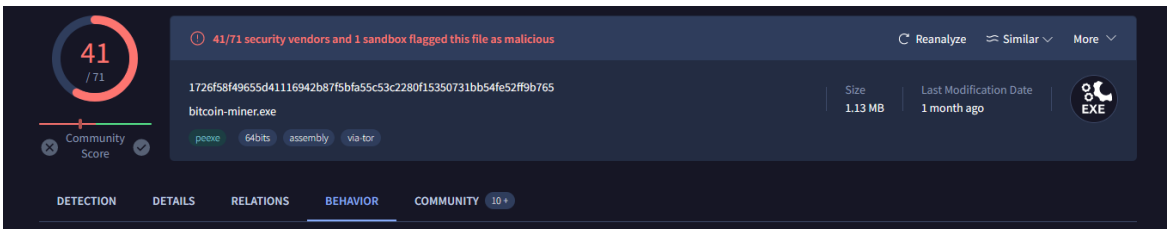

Figure 37: Technique 20 - Virus Total report 1

There is no capabilities section for this malware in VirusTotal.

A.1.21. Technique 21

The YARA rule will detect the malware if the YARA finds the string LocalSize

```
rule Detect_LocalSize: AntiDebug {
    meta:
        description = "Detect LocalSize as anti-debug"
        author = "Unprotect"
        comment = "Experimental rule"
    strings:
        $1 = "LocalSize" fullword ascii
    condition:
        $1
}
```

After choosing randomly one of the hashes between all the detected virus we search for it in VirusTotal trying to contrast the information.

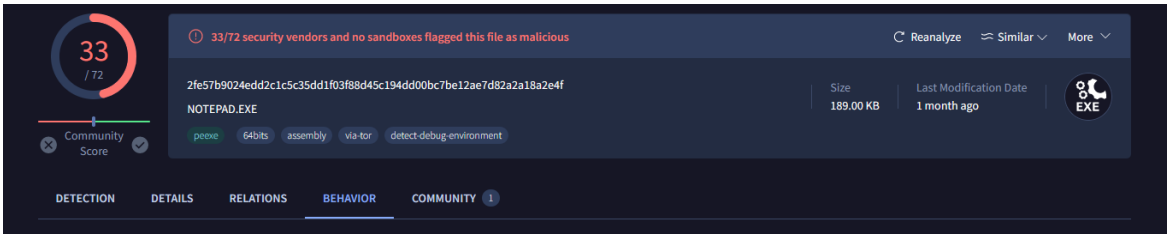

Figure 38: Technique 21 - Virus Total report 1

There is no capabilities section for this malware in VirusTotal.

A.1.22. Technique 22

The YARA rule will detect the malware if the YARA finds the strings used to check the use of a debugger via API.

```
rule debugger_via_API {
    meta:
        name = "check for debugger via API"
        author = "Teo-Prats"
        comment = "Converted from CAPA to YARA rule"
        reference = "https://github.com/LordNoteworthy/al-
khaser/blob/master/al-khaser/AntiDebug/CheckRemoteDebuggerPresent.cpp |
michael.hunhoff@fireeye.com"
    strings:
        $api1 = "CheckRemoteDebuggerPresent"
        $api2 = "WudfIsAnyDebuggerPresent"
        $api3 = "WudfIsKernelDebuggerPresent"
        $api4 = "WudfIsUserDebuggerPresent"
    condition:
        any of ($api*)
}
```

The only detected malware appears in VirusTotal as 0 flagged as malicious by the vendors.

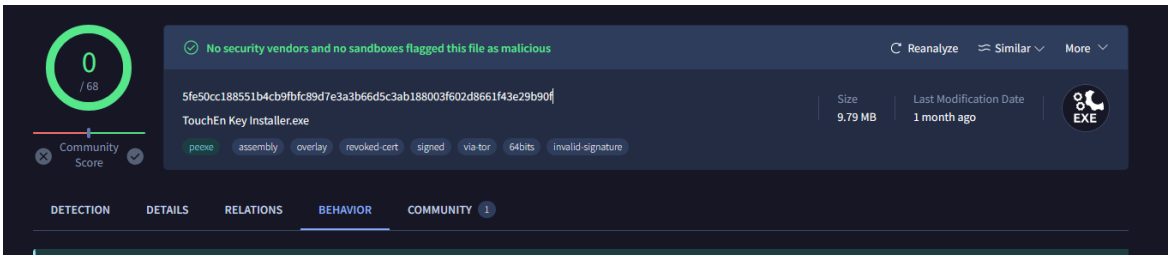

Figure 39: Technique 22 - Virus Total report 1

In the capabilities section is not listed any Anti-debugging method so it is proved again the effectiveness of this experimental rules.

Figure 40: Technique 22 - Virus Total report 2

A.1.23. Technique 23

The YARA rule will detect the malware if the YARA finds the strings used to check flag BeingDebugged.

```
rule PEB_BeingDebugged_flag {
    meta:
        name = "check for PEB BeingDebugged flag"
        author = "Teo-Prats"
        comment = "Converted from CAPA to YARA rule"
        references = "Practical Malware Analysis, Chapter 16, p. 353   |
moritz.raabe@fireeye.com"
    strings:
        $peb_access = { 64 A1 30 00 00 00 }
        $being_debugged_check = { 8B 40 02  84 C0 75 ?? }
    condition:
        $peb_access and $being_debugged_check
}
```

Unfortunately, this rule did not detect any malware sample so no testing has been performed.

A.1.24. Technique 24

The YARA rule will detect the malware if the YARA finds the string GetTickCount so detects that the function is being imported and the call to GetTickCount function.

```
rule GetTickCount {
    meta:
        name = "check for time delay via GetTickCount"
        author = "Teo-Prats"
        comment = "Converted from CAPA to YARA rule"
        references = "Practical Malware Analysis, Chapter 16, p. 353   |
michael.hunhoff@fireeye.com"
    strings:
        $GetTickCount = { FF 15 ?? ?? ?? ?? }
        $GetTickCountString = "GetTickCount" fullword ascii
    condition:
        $GetTickCountString and #GetTickCount >= 2
}
```

After choosing randomly one of the hashes between all the detected virus we search for it in VirusTotal trying to contrast the information.

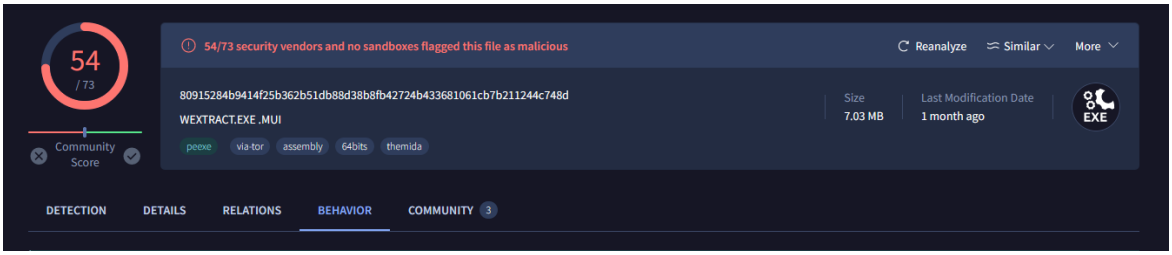

Figure 41: Technique 24 - Virus Total report 1

In the capabilities section is not listed any Anti-debugging method so it is proved again the effectiveness of this experimental rules.

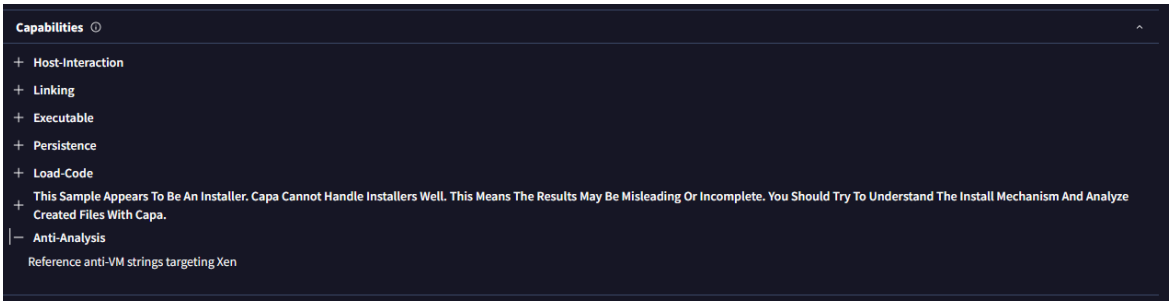

Figure 42: Technique 24 - Virus Total report 2

A.1.25. Technique 25

The YARA rule will detect the malware if the YARA finds the strings related to the interaction with the register DR0.

```
rule CheckForHardwareBreakpoints {
    meta:
        name = "check for hardware breakpoints"
        author = "Teo-Prats"
        comment = "Converted from CAPA to YARA rule"
        reference = "https://github.com/LordNoteworthy/al-
khaser/blob/master/al-khaser/AntiDebug/HardwareBreakpoints.cpp |
michael.hunhoff@fireeye.com"
    strings:
        $GetThreadContext = "GetThreadContext"
        $CONTEXT_DEBUG_REGISTERS = { 10 00 01 00 }
        $DR0_access = { c7 45 ?? 00 00 00 00 }
        $cmp_instr1 = { 3b c0 }
        $cmp_instr2 = { 3b c1 }
        $cmp_instr3 = { 3b c2 }
        $cmp_instr4 = { 3b c3 }
    condition:
        $GetThreadContext and $CONTEXT_DEBUG_REGISTERS and
        $DR0_access and (4 of ($cmp_instr*))
}
```

After choosing randomly one of the hashes between all the detected virus we search for it in VirusTotal trying to contrast the information.

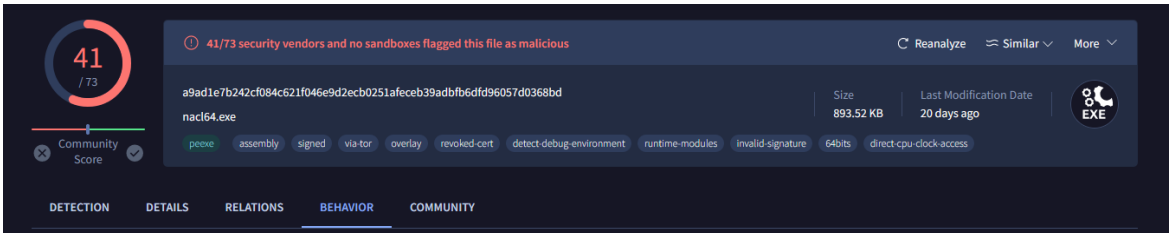

Figure 43: Technique 25 - Virus Total report 1

In the capabilities section is not listed any Anti-debugging method so it is proved again the effectiveness of this experimental rules.

Figure 44: Technique 25 - Virus Total report 2

A.1.26. Technique 26

The YARA rule will detect the malware if the YARA finds the strings that checks if the register EFLAGS has been manipulated and the strings that checks if the Trap Flag has been manipulated.

```
rule CheckForTrapFlagException {
    meta:
        name = "check for trap flag exception"
        author = "Teo-Prats"
        comment = "Converted from CAPA to YARA rule"
        reference = "https://github.com/LordNoteworthy/al-
khaser/blob/master/al-khaser/AntiDebug/TrapFlag.cpp |
michael.hunhoff@mandiant.com"
    strings:
        $pushf = { 9C }
        $popf = { 9D }
        $pushfd = { 66 9C }
        $popfd = { 66 9D }
        $pushfq = { 48 9C }
        $popfq = { 48 9D }
        $or_trap_flag = { 0B 24 ?? 00 01 }
        $bts_trap_flag = { 0F AB ?? 08 }
    condition:
        (any of ($pushf, $pushfd, $pushfq)) and
        (any of ($popf, $popfd, $popfq)) and
        (any of ($or_trap_flag, $bts_trap_flag))
}
```

After choosing randomly one of the hashes between all the detected virus we search for it in VirusTotal trying to contrast the information.

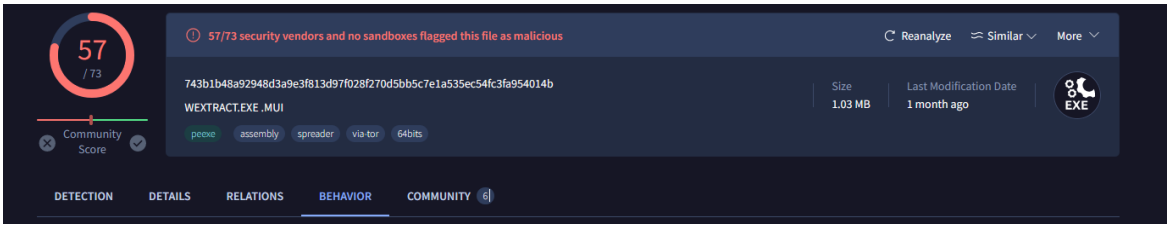

Figure 45: Technique 26 - Virus Total report 1

In the capabilities section is not listed any Anti-debugging method so it is proved again the effectiveness of this experimental rules.

Figure 46: Technique 26 - Virus Total report 2

A.1.27. Technique 27

The YARA rule will detect the malware if the YARA finds the strings to call the function NtYieldExecution or similar like the SwitchToThread then checks the STATUS_NO_YIELD_PERFORMED value.

```
rule NtYieldExecution_SwitchToThread_AntiDebug
{
    meta:
        description = "Detects the use of NtYieldExecution or
SwitchToThread for anti-debugging purposes."
        author = "Teo-Prats"
        reference= "https://anti-debug.checkpoint.com/techniques/misc.html"
    strings:
        $nt_yield_execution = "NtYieldExecution"
        $switch_to_thread = "SwitchToThread"
        $status_no_yield_performed = { 24 00 00 40 }
    condition:
         $nt_yield_execution or $switch_to_thread or
$status_no_yield_performed

}
```

After choosing randomly one of the hashes between all the detected virus we search for it in VirusTotal trying to contrast the information.

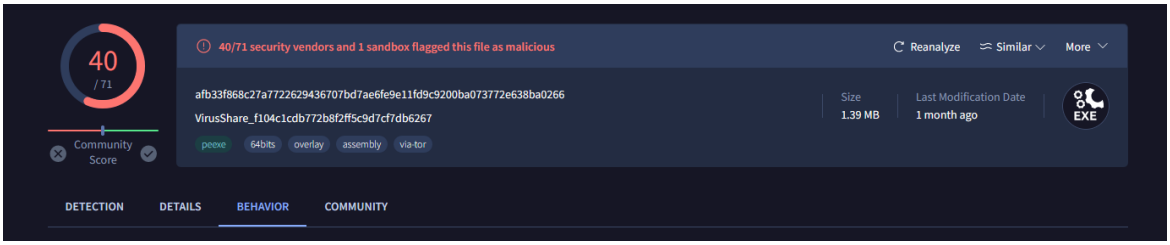

Figure 47: Technique 27 - Virus Total report 1

There is no capabilities section for this malware in VirusTotal.

A.1.28. Technique 28

The YARA rule will detect the malware if the YARA finds the strings VirtualAlloc and GetWriteWatch commonly used to track debuggers that may modify memory pages outside the expected pattern.

```
rule VirtualAlloc_GetWriteWatch {
    meta:
        description = "Detects the use of VirtualAlloc and GetWriteWatch
functions"
        author = "Teo-Prats"
        reference= "https://anti-debug.checkpoint.com/techniques/misc.html"
    strings:
        $virtualalloc = "VirtualAlloc" nocase wide ascii
        $getwritewatch = "GetWriteWatch" nocase wide ascii
    condition:
        any of them
}
```

After choosing randomly one of the hashes between all the detected virus we search for it in VirusTotal trying to contrast the information.

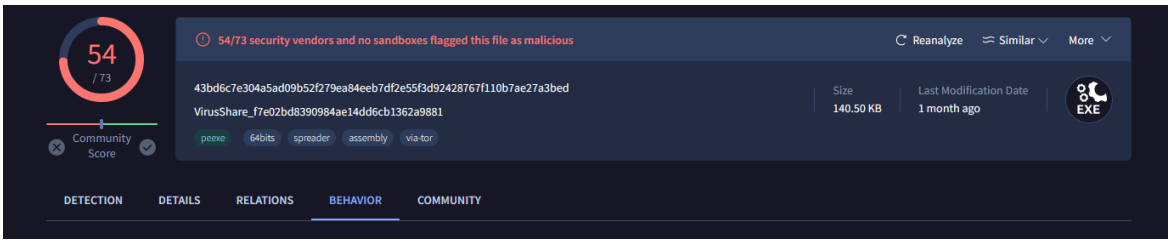

Figure 48: Technique 28 - Virus Total report 1

A.1.29. Technique 29

The YARA rule will detect the malware if the YARA finds the string opcode for the INT3 + RET commonly used to insert interruption points and search for common sequences of the SEH (Structured Exception Handling)

```
rule CheckForInt3RetAntiDebug {
    meta:
        description = "Detects INT 3 and RET instructions used in a try-except block for anti-debugging"
        author = "Teo-Prats"
        reference = "https://unprotect.it/technique/call-to-interrupt-procedure/"

    strings:
        $int3_ret = { CD 03 C3 }
        $seh_prologue = { 64 A1 00 00 00 00 50 64 89 25 00 00 00 00 }
        $seh_epilogue = { 64 89 0D 00 00 00 00 58 64 A3 00 00 00 00 }

    condition:
        $int3_ret and
        any of ($seh_prologue, $seh_epilogue)
}
```

After choosing randomly one of the hashes between all the detected virus we search for it in VirusTotal trying to contrast the information.

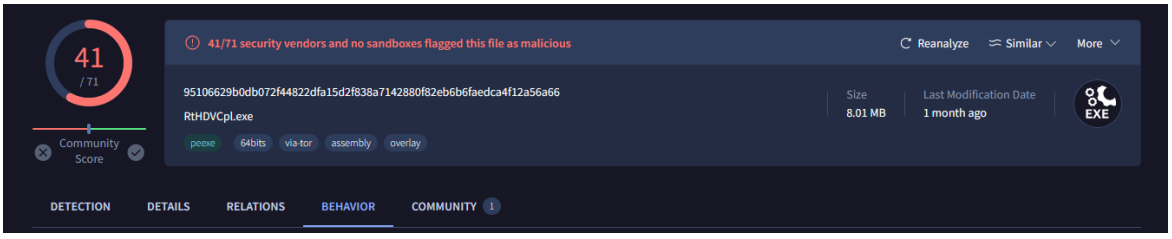

Figure 49: Technique 29 - Virus Total report 1

In the capabilities section is not listed any Anti-debugging method so it is proved again the effectiveness of this experimental rules.

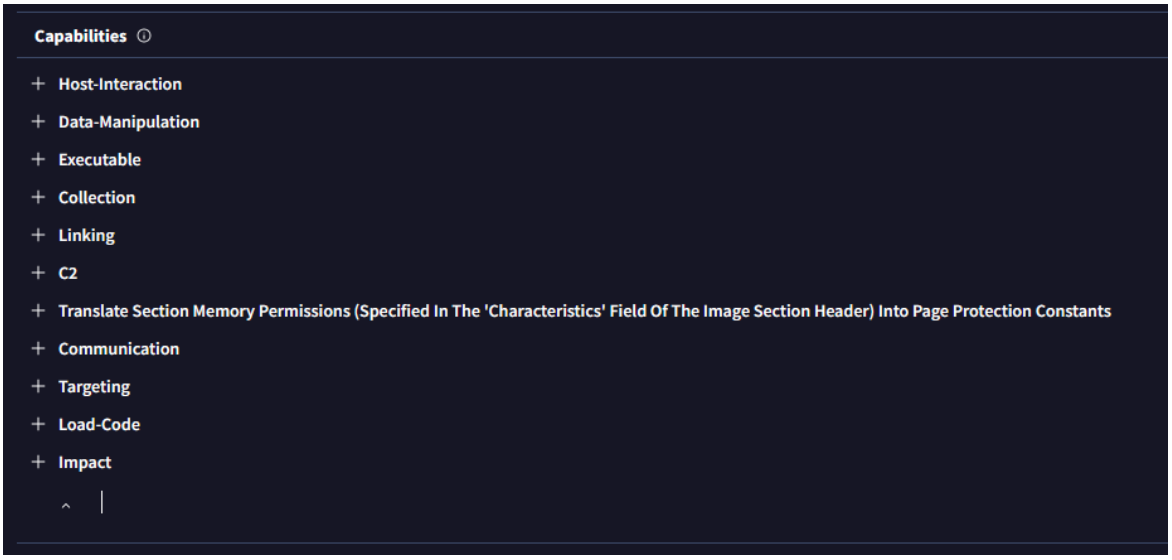

Figure 50: Technique 29 - Virus Total report 2

A.1.30. Technique 30

The YARA rule will detect the malware if the YARA finds the strings RaiseException and the opcode for the value DBG_PRINTEXCEPTION_C

```
rule DbgPrint_AntiDebug
{
    meta:
        description = "Detects the use of RaiseException with
DBG_PRINTEXCEPTION_C for anti-debugging purposes."
        author = "Teo-Prats"
        reference ="https://anti-debug.checkpoint.com/techniques/misc.html"

    strings:
        $raise_exception = "RaiseException"
        $dbg_printexception_c = { 06 00 01 40 }
    condition:
        $raise_exception and $dbg_printexception_c
}
```

After choosing randomly one of the hashes between all the detected virus we search for it in VirusTotal trying to contrast the information.

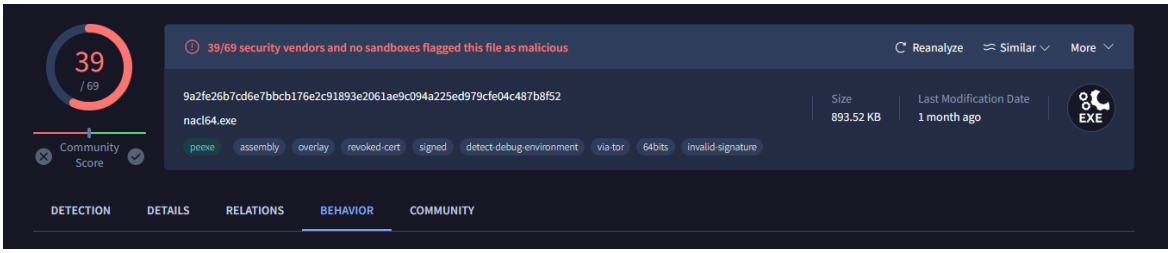

Figure 51: Technique 30 - Virus Total report 1

In the capabilities section is not listed any Anti-debugging method so it is proved again the effectiveness of this experimental rules.

Figure 52: Technique 30 - Virus Total report 2

A.1.31. Technique 31

The YARA rule will detect the malware if the YARA finds the strings typically used for download payloads.

```
rule FuncIn {
    meta:
        description = "Detects malware loaders that download payloads from
a C2 server"
        author = "Teo-Prats"
        reference = "https://unprotect.it/technique/funcin/"

    strings:
        $network_communication = { 68 ?? ?? ?? ?? E8 ?? ?? ?? ?? 83 C4 04 }
        $http_request = "GET / HTTP/1.1" ascii
        $dns_api = "DnsQuery_A" ascii
        $load_library = "LoadLibrary" ascii
        $get_proc_address = "GetProcAddress" ascii
        $virtual_alloc = "VirtualAlloc" ascii
        $write_process_memory = "WriteProcessMemory" ascii
        $create_remote_thread = "CreateRemoteThread" ascii

    condition:
        any of ($network_communication, $http_request, $dns_api,
$load_library, $get_proc_address, $virtual_alloc, $write_process_memory,
$create_remote_thread)
}
```

After choosing randomly one of the hashes between all the detected virus we search for it in VirusTotal trying to contrast the information.

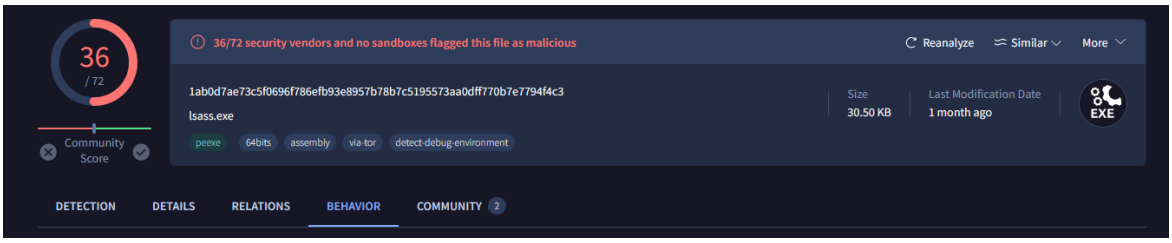

Figure 53: Technique 31 - Virus Total report 1

There is no capabilities section for this malware in VirusTotal.

A.1.32. Technique 32

The YARA rule will detect the malware if the YARA finds the strings of the RltQueryProcessHeapInformation function and heap flags manipulation.

```
rule RtlQueryProcessHeapInformation {
    meta:
        description = "Detects the use of RtlQueryProcessHeapInformation
and related heap flag manipulations for anti-debugging"
        author = "Teo-Prats"
        reference = "https://anti-debug.checkpoint.com/techniques/debug-
flags.html#using-win32-api-ntqueryinformationprocess"

    strings:
        $rtl_query_heap = "RtlQueryProcessHeapInformation" ascii
        $ntdll = "ntdll.dll" ascii
        $heap_flags_check = { 0F B7 45 ?? 3D ?? ?? 00 00 }
        $heap_flags_set = { C7 45 ?? ?? ?? ?? 00 00 }

    condition:

        all of ($rtl_query_heap, $ntdll) or any of ($heap_flags_check,
$heap_flags_set)
}
```

After choosing randomly one of the hashes between all the detected virus we search for it in VirusTotal trying to contrast the information.

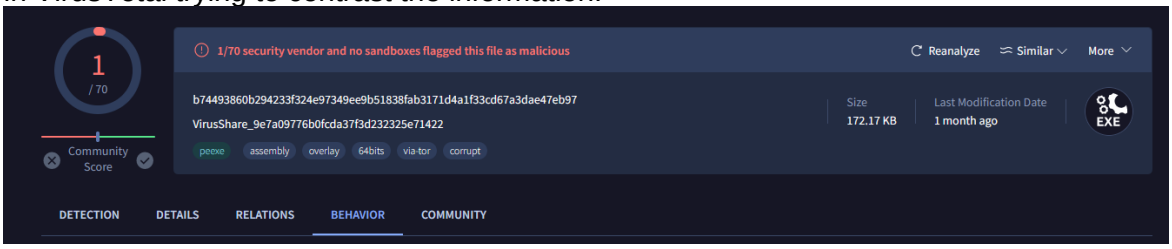

Figure 54: Technique 32 - Virus Total report 1

In the capabilities section is not listed any Anti-debugging method so it is proved again the effectiveness of this experimental rules.

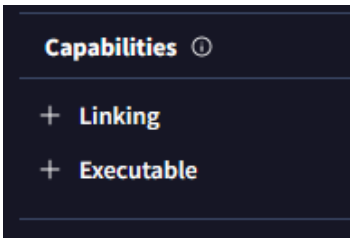

Figure 55: Technique 32 - Virus Total report 2

A.1.33. Technique 33

The YARA rule will detect the malware if the YARA finds the strings related to check the flag BeingDebugged in the PEB, for 32 and 64 bits.

```
rule DetectPEBBeingDebuggedFlagCheck {
    meta:
        name = "Detect PEB BeingDebugged Flag Check"
        description = "Detects reading the BeingDebugged flag from the PEB
for anti-debugging"
        author = "Teo-Prats"
        reference = "https://anti-debug.checkpoint.com/techniques/debug-
flags.html#manual-checks-peb-beingdebugged-flag"
    strings:
        $fsdword = { 64 A1 30 00 00 00 }
        $being_debugged_32 = { 64 A1 30 00 00 00 8A 40 02 }

        $gsqword = { 65 48 8B 04 25 60 00 00 00 }
        $being_debugged_64 = { 65 48 8B 04 25 60 00 00 00 0F B6 40 02 }
    condition:
        any of ($fsdword, $being_debugged_32, $gsqword, $being_debugged_64)
}
```

After choosing randomly one of the hashes between all the detected virus we search for it in VirusTotal trying to contrast the information.

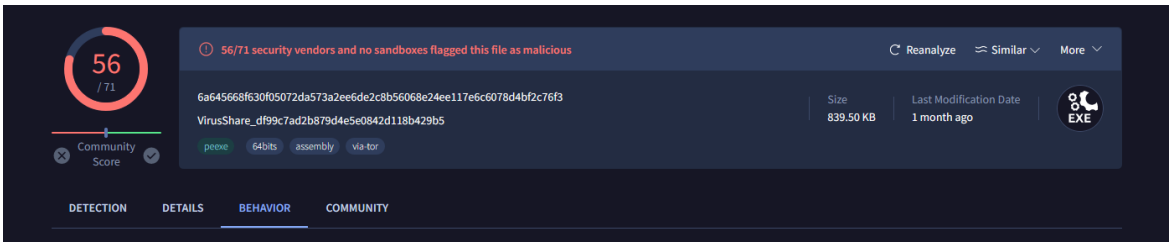

Figure 56: Technique 33 - Virus Total report 1

In the capabilities section there are an anti-debugging method listed but it hardly is checking the same as this experimental rule.

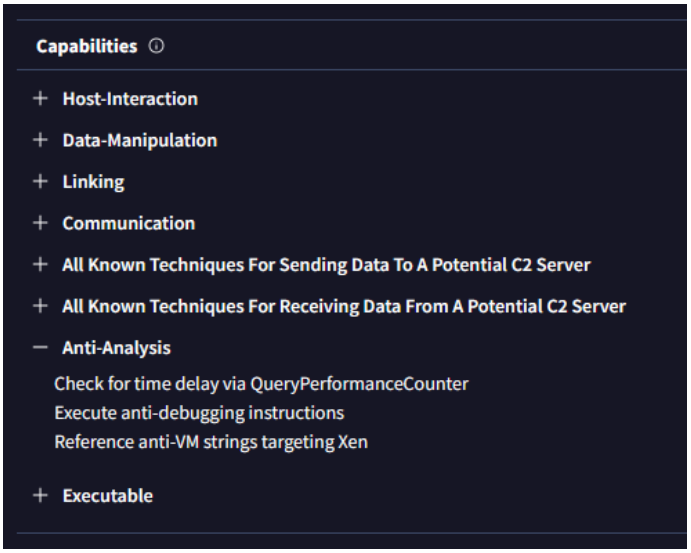

Figure 57: Technique 33 - Virus Total report 2

A.1.34. Technique 34

The YARA rule will detect the malware if the YARA finds the strings checking sequences relative to checking the heap tail and heap free checking.

```
rule DetectHeapTailAndFreeCheckingFlags {
    meta:
        description = "Detects the presence of 0xABABABAB and 0xFEEEFEEE
sequences indicative of heap tail and free checking flags"
        author = "Teo-Prats"
        reference = "o  https://anti-debug.checkpoint.com/techniques/debug-
flags.html#manual-checks-heap-protection"

    strings:
        $heap_tail_checking_32 = { AB AB AB AB AB AB AB AB }
        $heap_tail_checking_64 = { AB AB AB AB AB AB AB AB AB AB AB AB AB
AB AB AB }
        $heap_free_checking = { FE EE FE EE }

    condition:
        any of ($heap_tail_checking_32, $heap_tail_checking_64,
$heap_free_checking)
}
```

After choosing randomly one of the hashes between all the detected virus we search for it in VirusTotal trying to contrast the information.

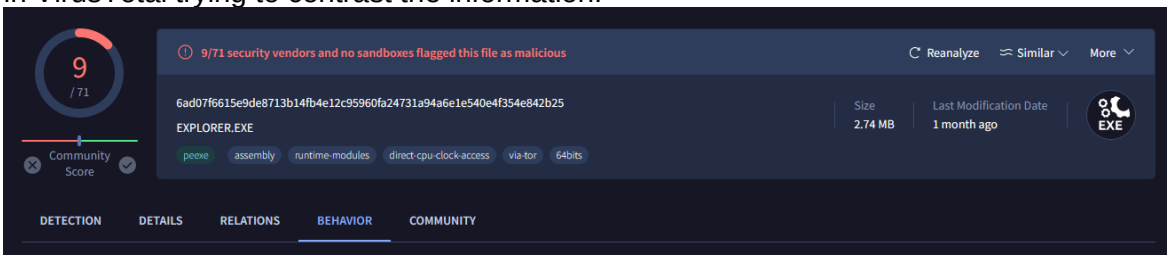

Figure 58: Technique 34 - Virus Total report 1

There is no capabilities section for this malware in VirusTotal.

A.1.35. Technique 35

The YARA rule will detect the malware if the YARA finds the string used to call the function SwitchDesktop and the sequence related to a event desktop.

```
rule detect_desktop_switching {
    meta:
        description = "Detects potential desktop switching to evade
debugging"
        author = "TeoPrats"
        reference= "https://anti-
debug.checkpoint.com/techniques/interactive.html"
    strings:
        $desktop_check = "SwitchDesktop"
        $desktop_switch_api = "user32.dll"
        $desktop_event_check = /[\x10-
\x1F]\x00\x00\x00..\x00\x00\x00\x00..\x00\x00\x00\x00\x00/
    condition:
        any of ($desktop_check, $desktop_switch_api) and
$desktop_event_check
}
```

After choosing randomly one of the hashes between all the detected virus we search for it in VirusTotal trying to contrast the information.

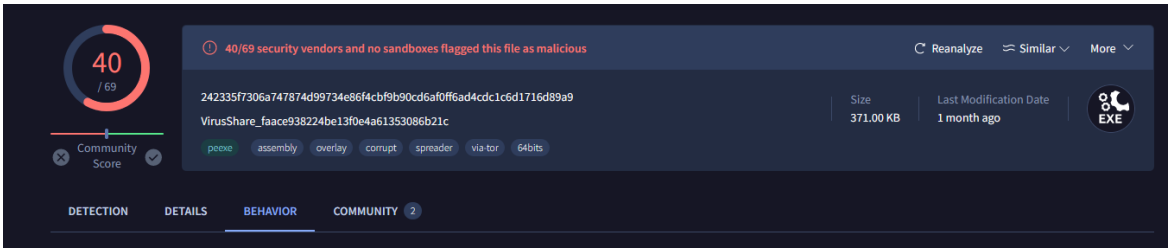

Figure 59: Technique 35 - Virus Total report 1

In the capabilities section is not listed any Anti-debugging method so it is proved again the effectiveness of this experimental rules.

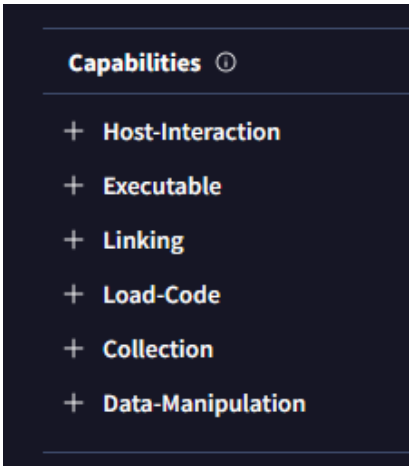

Figure 60: Technique 35 - Virus Total report 2

A.1.36. Technique 36

The YARA rule will detect the malware if the YARA finds the string OpenProcess used to open the critical process csrss.exe used to check if the environment is being debugged.

```
rule detect_OpenProcess{
    meta:
        description = "Detects calls to OpenProcess on csrss.exe
potentially for debugging detection"
        author = "Teo-Prats"
        reference = "https://anti-debug.checkpoint.com/techniques/object-
handles.html#openprocess"
    strings:
        $openProcess = "OpenProcess" wide ascii
        $csrss = "csrss.exe"
    condition:
        all of ($openProcess, $csrss)
}
```

After choosing randomly one of the hashes between all the detected virus we search for it in VirusTotal trying to contrast the information. In this case no security vendor has flagged this file as malicious.

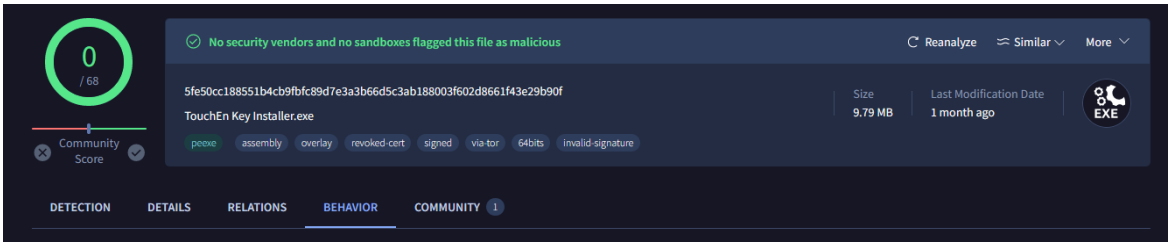

Figure 61: Technique 36 - Virus Total report 1

Anyway, in the capabilities section we still can see how the file uses techniques related to malware, in this case is not listed any Anti-debugging method so it is proved again the effectiveness of this experimental rules.

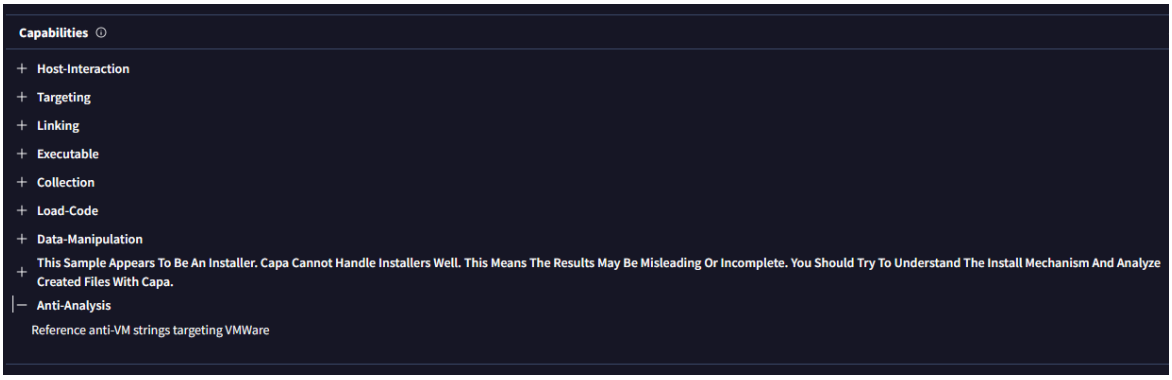

Figure 62: Technique 36 - Virus Total report 2

A.1.37. Technique 37

The YARA rule will detect the malware if the YARA finds the strings to detect the use of the functions CreateFileW and CreateFileA to open process files.

```
rule DetectCreateFile {
    meta:
        description = "Detects the use of CreateFileW/CreateFileA to
exclusively open the process file to detect debuggers"
        author = "Teo-Prats"
        reference = "https://anti-debug.checkpoint.com/techniques/object-
handles.html#createfile"

    strings:
        $CreateFileW = "CreateFileW"
        $CreateFileA = "CreateFileA"
        $currentProcess = "\\Device\\HarddiskVolume" wide ascii

    condition:
        any of ($CreateFileW, $CreateFileA) and $currentProcess
}
```

This sample is the same as the previous one, that is because both rules only found a sample and it was the same, probably because both rules used a similar approach or it was too generic.

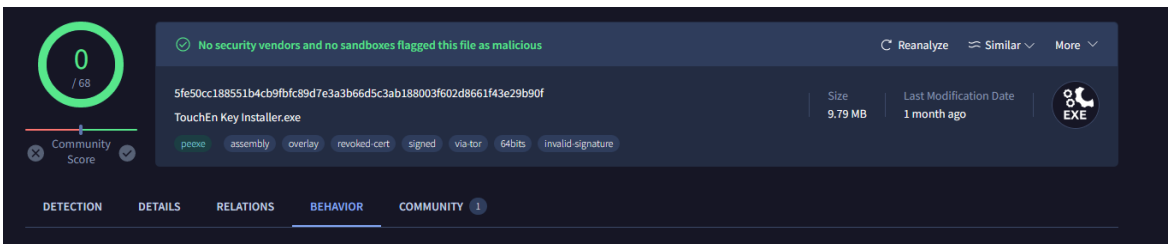

Figure 63: Technique 37 - Virus Total report 1

In the capabilities section is not listed any Anti-debugging method so it is proved again the effectiveness of this experimental rules.

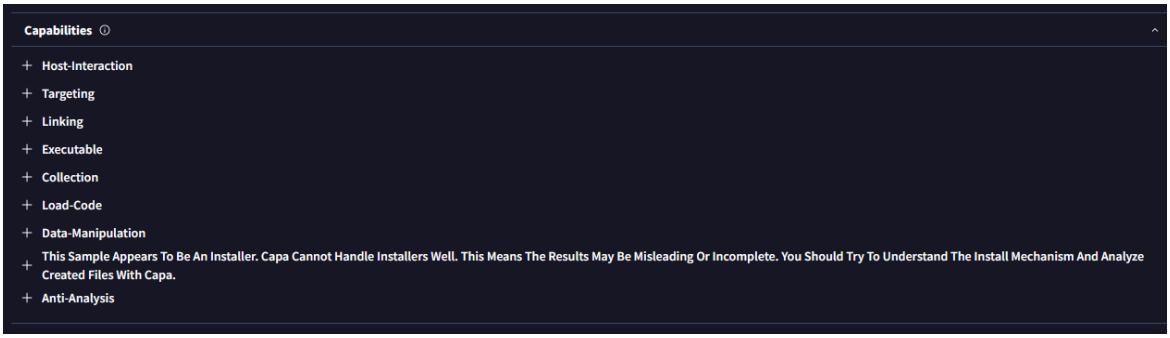

Figure 64: Technique 37 - Virus Total report 2

A.1.38. Technique 38

The YARA rule will detect the malware if the YARA finds the string to load a library and then create a file to check the presence of a debugger.

```
rule DetectLoadLibraryCreateFileCheck {
    meta:
        description = "Detects the use of LoadLibraryA/W followed by
CreateFileA/W to check for the presence of a debugger"
        author = "Teo-Prats"
        reference = "https://anti-debug.checkpoint.com/techniques/object-
handles.html#loadlibrary"
    strings:
        $LoadLibraryA = "LoadLibraryA"
        $LoadLibraryW = "LoadLibraryW"
        $CreateFileA = "CreateFileA"
        $CreateFileW = "CreateFileW"
    condition:
        ($LoadLibraryA and $CreateFileA)
         or
        ($LoadLibraryW and $CreateFileW)
}
```

After choosing randomly one of the hashes between all the detected virus we search for it in VirusTotal trying to contrast the information.

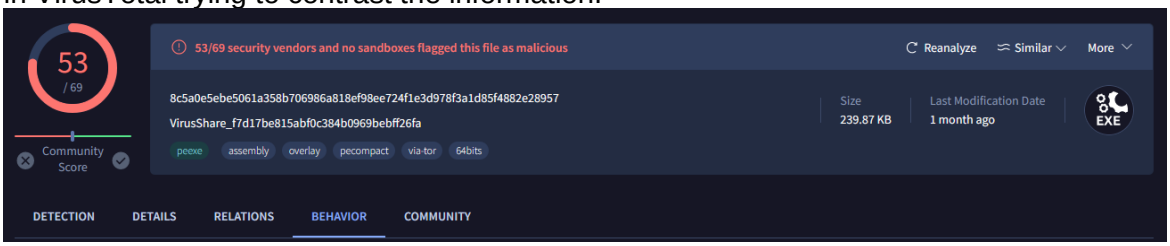

Figure 65: Technique 38 - Virus Total report 1

In the capabilities section is not listed any Anti-debugging method so it is proved again the effectiveness of this experimental rules.

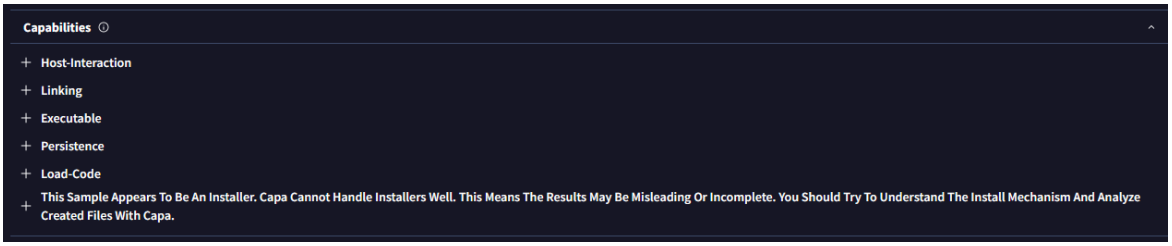

Figure 66: Technique 38 - Virus Total report 2

A.1.39. Technique 39

The YARA rule will detect the malware if the YARA finds the string to use the function RaiseException and then the check of the values DBG_CONTROL_C and DBG_RIPEVENT.

```
rule DetectRaiseExceptionDebuggerCheck {
    meta:
        description = "Detects the use of RaiseException to raise
exceptions like DBG_CONTROL_C or DBG_RIPEVENT to check for the presence of
a debugger"
        author = "Teo-Prats"
        reference = "https://anti-
debug.checkpoint.com/techniques/exceptions.html#raiseexception"
    strings:
        $RaiseException = "RaiseException"
        $DBG_CONTROL_C = { 1D 00 00 00 }
        $DBG_RIPEVENT = { 2D 00 00 00 }
    condition:
        $RaiseException and ($DBG_CONTROL_C or $DBG_RIPEVENT)
}
```

After choosing randomly one of the hashes between all the detected virus we search for it in VirusTotal trying to contrast the information.

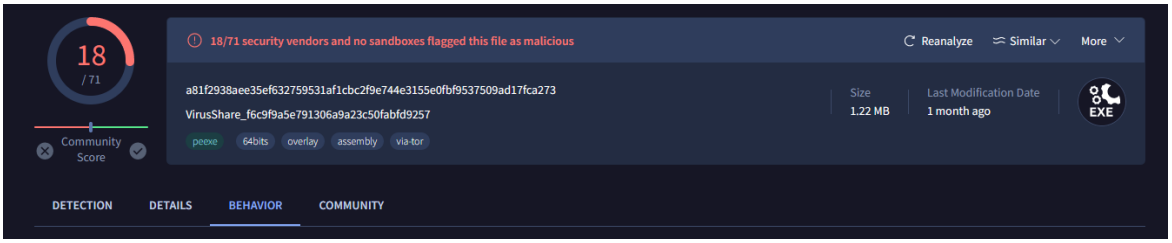

Figure 67: Technique 39 - Virus Total report 1

In the capabilities section is not listed any Anti-debugging method so it is proved again the effectiveness of this experimental rules.

Figure 68: Technique 39 - Virus Total report 2

A.1.40. Technique 40

The YARA rule will detect the malware if the YARA finds the strings to check the use of the exception handlers.

```
rule DetectControlFlowHidingWithExceptions {
    meta:
        description = "Detects the use of a sequence of exception handlers
to hide control flow"
        author = "Teo-Prats"
        reference = "https://anti-
debug.checkpoint.com/techniques/exceptions.html"
    strings:
        $AddVectoredExceptionHandler = "AddVectoredExceptionHandler"
        $SetUnhandledExceptionFilter = "SetUnhandledExceptionFilter"
        $RaiseException = "RaiseException"
        $try = "__try"
        $except = "__except"
    condition:
        ($AddVectoredExceptionHandler and $RaiseException) or
        ($SetUnhandledExceptionFilter and $RaiseException) or
        ($try and $except and $RaiseException)
}
```

After choosing randomly one of the hashes between all the detected virus we search for it in VirusTotal trying to contrast the information.

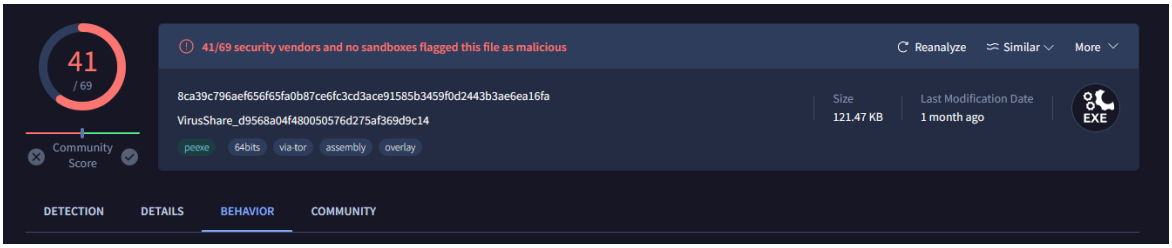

Figure 69: Technique 40 - Virus Total report 1

In the capabilities section is not listed any Anti-debugging method so it is proved again the effectiveness of this experimental rules.

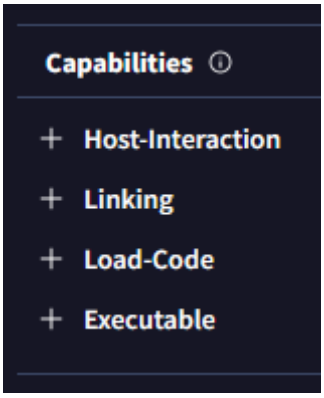

Figure 70: Technique 40 - Virus Total report 2

A.1.41. Technique 41

The YARA rule will detect the malware if the YARA finds the strings ZwGetTickCount and KiGetTickCount to detect the use of the functions and the kusher shared data sequence which is a characteristic of direct access to shared system data.

```
rule DetectZwGetTickCount{
    meta:
        description = "Detects the usage of ZwGetTickCount() or direct
reads from KUSER_SHARED_DATA as anti-debugging techniques"
        author = "Teo-prats"
        reference = "https://anti-
debug.checkpoint.com/techniques/timing.html"

    strings:
        $ZwGetTickCount = "ZwGetTickCount"
        $KiGetTickCount = "KiGetTickCount"
        $KUSER_SHARED_DATA = { 7F FE 00 00 }
    condition:
        $ZwGetTickCount or
        $KiGetTickCount or
        $KUSER_SHARED_DATA
}
```

After choosing randomly one of the hashes between all the detected virus we search for it in VirusTotal trying to contrast the information.

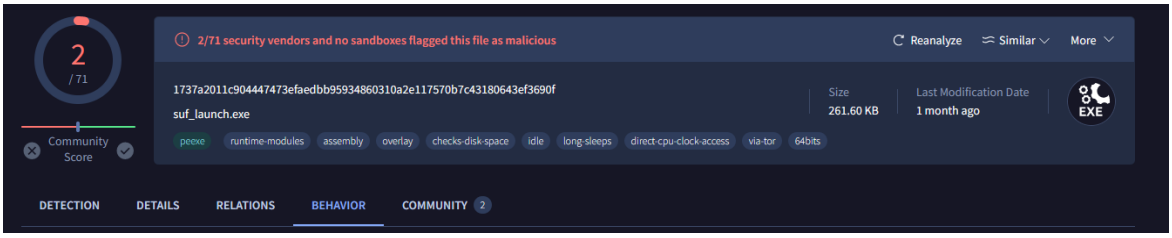

Figure 71: Technique 41 - Virus Total report 1

In the capabilities section is not listed any Anti-debugging method so it is proved again the effectiveness of this experimental rules.

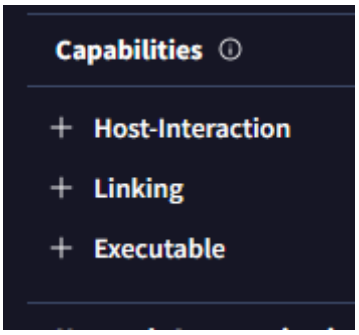

Figure 72: Technique 41 - Virus Total report 2

A.1.42. Technique 42

The YARA rule will detect the malware if the YARA finds the string BlockInput used to block user input.

```
rule BlockInput_AntiDebug
{
    meta:
        description = "Detects the use of BlockInput for anti-debugging
purposes."
        author = "TeoPrats"
        reference= "https://anti-
debug.checkpoint.com/techniques/interactive.html"

    strings:
        $block_input = "BlockInput"
    condition:
        any of them
}
```

After choosing randomly one of the hashes between all the detected virus we search for it in VirusTotal trying to contrast the information.

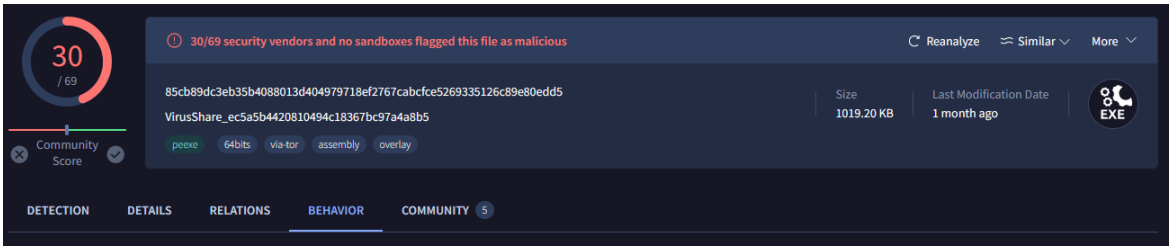

Figure 73: Technique 42 - Virus Total report 1

In the capabilities section is not listed any Anti-debugging method similar to this rule, only the check for time delay so it is proved again the effectiveness of this experimental rules.

Figure 74: Technique 42 - Virus Total report 2

A.1.43. Technique 43

The YARA rule will detect the malware if the YARA finds the opcodes sequences which means the selector is being manipulated.

```
rule Selector_Manipulation_AntiDebug
{
    meta:
        description = "Detects the use of selector manipulation for anti-debugging purposes."
        author = "Teo-Prats"
        reference="https://anti-debug.checkpoint.com/techniques/misc.html"
    strings:
        $xor_eax_eax = { 31 C0 }
        $push_fs = { 0F A0 }
        $pop_ds = { 1F }
        $xchg_eax_cl = { 86 08 }
        $int3 = { CC }
        $push_offset = { 68 ?? ?? ?? ?? }
        $pop_gs = { 0F A9 }
        $mov_fs_eax = { 64 89 20 }
        $cmp_al_3 = { 3C 03 }
        $je = { 74 ?? }
    condition:
        all of ($xor_eax_eax, $push_fs, $pop_ds, $xchg_eax_cl) or
        all of ($push_offset, $int3, $pop_gs, $mov_fs_eax, $cmp_al_3, $je)
 }
```

After choosing randomly one of the hashes between all the detected virus we search for it in VirusTotal trying to contrast the information.

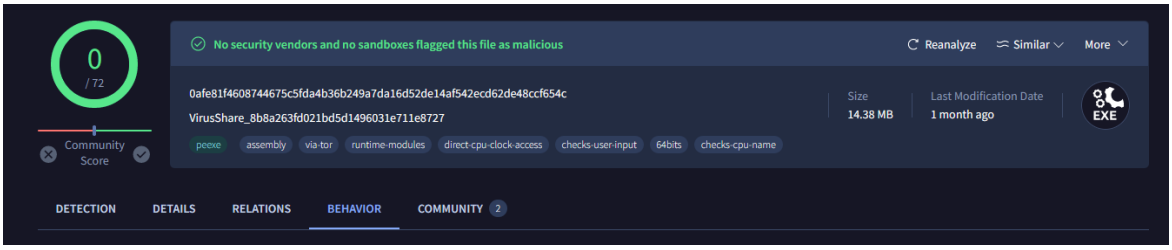

Figure 75: Technique 43 - Virus Total report 1

There is no capabilities section for this malware in VirusTotal.

A.1.44. Technique 44

The YARA rule will detect the malware if the YARA finds the opcode that represents push ss, pop ss, and pushf instructions.

```
rule StackSegmentRegister {
    meta:
        description = "Detects the anti-debugging technique that checks for
the Trap Flag by using push ss, pop ss, and pushf instructions"
        author = "Teo-Prats"
        reference = "https://anti-
debug.checkpoint.com/techniques/assembly.html"
    strings:
        $sequence = { 16 1F 9C }
    condition:
        $sequence
}
```

After choosing randomly one of the hashes between all the detected virus we search for it in VirusTotal trying to contrast the information.

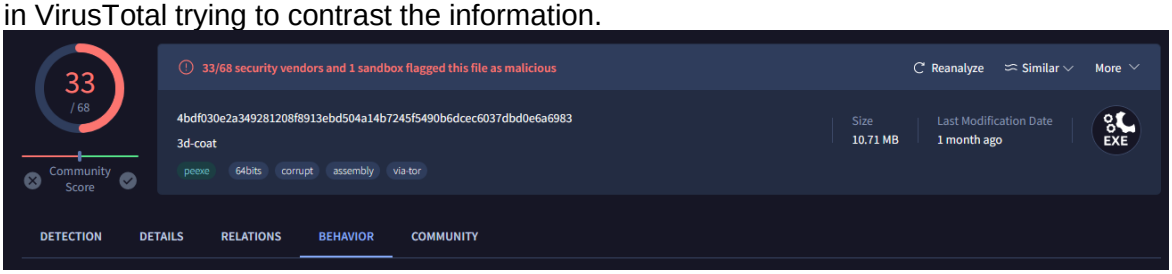

Figure 76: Technique 44 - Virus Total report 1

There is no capabilities section for this malware in VirusTotal.

A.1.45. Technique 45

The YARA rule will detect the malware if the YARA finds the specific byte sequence { F3 64 F1 }, which represents a series of instruction prefixes combined with the int1 instruction.

```
rule DetectInstructionPrefixesAntiDebug {
    meta:
        description = "Detects the anti-debugging technique that uses instruction prefixes"
        author = "Teo-Prats"
        reference = "https://anti-debug.checkpoint.com/techniques/assembly.html"
    strings:
        $prefix_int1_sequence = { F3 64 F1 }
    condition:
        $prefix_int1_sequence
}
```

In the capabilities section is not listed any Anti-debugging method so it is proved again the effectiveness of this experimental rules.

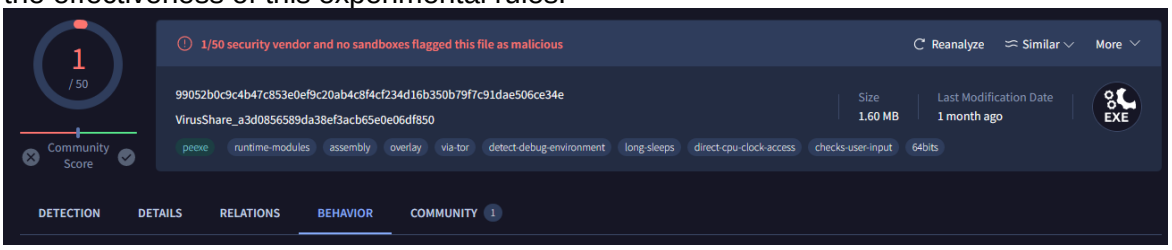

Figure 77: Technique 45 - Virus Total report 1

In the capabilities section is not listed any Anti-debugging method despite the check of the time delay so it is proved again the effectiveness of this experimental rules.

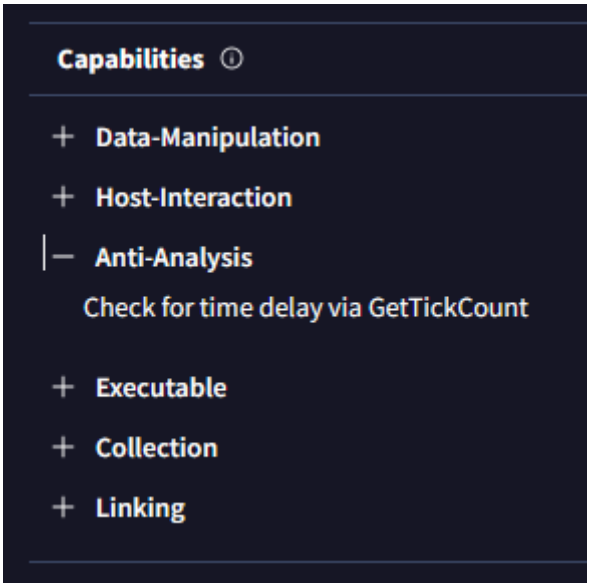

Figure 78: Technique 45 - Virus Total report 2

A.1.46. Technique 46

The YARA rule will detect the malware if the YARA finds the opcodes that corresponds to functions commonly used in self-debugging.

```
rule Self_Debugging_Detection
{
    meta:
        description = "Detects the use of self-debugging techniques in a
binary."
        author = "Teo-Prats"
        reference = "https://anti-
debug.checkpoint.com/techniques/interactive.html#self-debugging"
    strings:
        $debug_active_process = { 44 65 62 75 67 41 63 74 69 76 65 50 72 6F
63 65 73 73 }
        $create_event_w = { 43 72 65 61 74 65 45 76 65 6E 74 57
}
        $get_module_file_name_w = { 47 65 74 4D 6F 64 75 6C 65 46 69 6C 65
4E 61 6D 65 57 }
        $is_debugged_function = { 49 73 44 65 62 75 67 67 65 64
}
        $enable_debug_privilege = { 45 6E 61 62 6C 65 44 65 62 75 67 50 72
69 76 69 6C 65 67 65 }
    condition:
       any of($debug_active_process,
        $create_event_w,
        $get_module_file_name_w,
        $is_debugged_function,
        $enable_debug_privilege)
}
```

After choosing randomly one of the hashes between all the detected virus we search for it in VirusTotal trying to contrast the information. In this case no security vendor flagged this file as malicious.

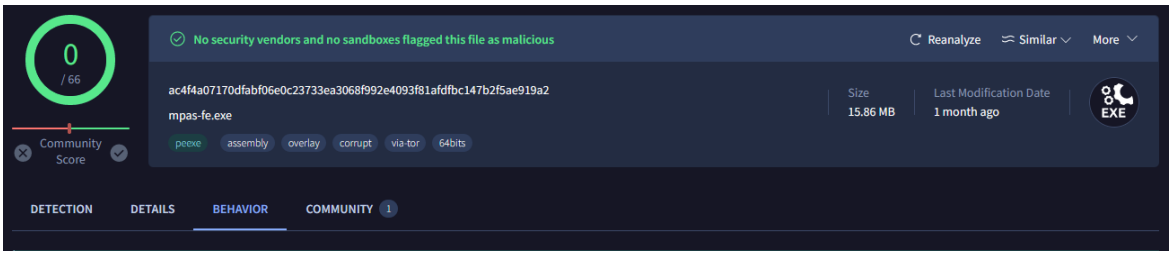

Figure 79: Technique 46 - Virus Total report 1

But we can appreciate in the capabilities section how the sample still use some techniques related to malware, again, it is not listed any Anti-debugging method so it is proved again the effectiveness of this experimental rules.

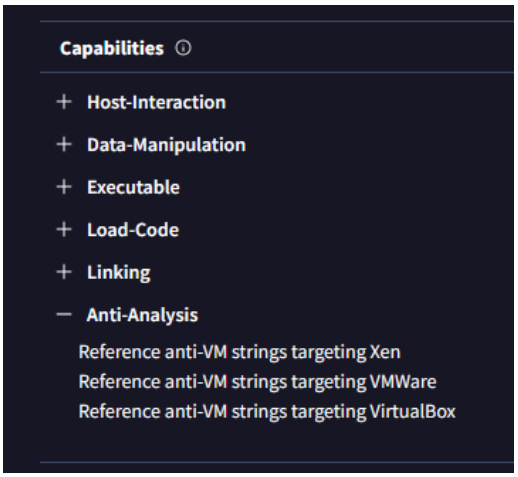

Figure 80: Technique 46 - Virus Total report 2

A.1.47. Technique 47

The YARA rule will detect the malware if the YARA finds the strings typically used in exception handlers and the function GenerateConsoleCtrlEvent.

```
rule GenerateConsoleCtrlEvent_AntiDebug
{
    meta:
        description = "Detects the use of GenerateConsoleCtrlEvent for
anti-debugging purposes."
        author = "Teo-Prats"
        reference = "https://anti-
debug.checkpoint.com/techniques/interactive.html"
    strings:
        $generate_console_ctrl_event = { 47 65 6E 65 72 61 74 65 43 6F 6E
73 6F 6C 65 43 74 72 6C 45 76 65 6E 74 }
        $dbg_control_c = { 44 42 47 5F 43 4F 4E 54 52 4F 4C 5F 43 }
        $add_vectored_exception_handler = { 41 64 64 56 65 63 74 6F 72 65
64 45 78 63 65 70 74 69 6F 6E 48 61 6E 64 6C 65 72 }
        $set_console_ctrl_handler = { 53 65 74 43 6F 6E 73 6F 6C 65 43 74
72 6C 48 61 6E 64 6C 65 72 }
        $remove_vectored_exception_handler = { 52 65 6D 6F 76 65 56 65 63
74 6F 72 65 64 45 78 63 65 70 74 69 6F 6E 48 61 6E 64 6C 65 72 }
    condition:
        $generate_console_ctrl_event or
        $dbg_control_c or
        $add_vectored_exception_handler or
        $set_console_ctrl_handler or
        $remove_vectored_exception_handler
}
```

After choosing randomly one of the hashes between all the detected virus we search for it in VirusTotal trying to contrast the information.

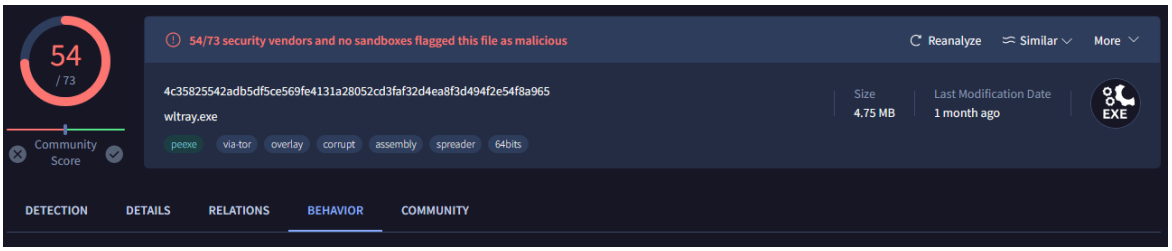

Figure 81: Technique 40 - Virus Total report 1

There is no capabilities section for this malware in VirusTotal.